

Building Applications with OpsDB Application Architecture

A Construction Reference for the OpsDB Application Platform

Registry: [\[HOWL-COMP-9-2026\]](#)

Series Path: [\[HOWL-COMP-1-2026\]](#) → [\[HOWL-COMP-2-2026\]](#) → [\[HOWL-COMP-3-2026\]](#) → [\[HOWL-COMP-4-2026\]](#) → [\[HOWL-COMP-5-2026\]](#) → [\[HOWL-COMP-6-2026\]](#) → [\[HOWL-COMP-7-2026\]](#) → [\[HOWL-COMP-8-2026\]](#) → [\[HOWL-COMP-9-2026\]](#)

DOI: 10.5281/zenodo.20061178

Date: May 2026

Domain: Software Architecture / Systems Engineering

Status: Architectural Blueprint for Independent Implementation

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet.

Abstract

OpsDB is a governed data substrate that unifies schema-driven validation, five-layer authorization, versioned state, change management with approval routing, append-only audit logging, structured search, and bounded automation into a single ten-step gate pipeline. Built in Go on Postgres and released under the MIT license, the compiled infrastructure contains zero application-domain types — all domain knowledge lives in YAML schema files and data rows interpreted by fixed mechanisms. An application built on OpsDB is called an AppDB. The developer writes schema YAML declaring entities, fields, types, constraints, and relationships. Running the schema loader produces a complete backend with validated writes, access control, version history, change management, audit trail, and search — without writing endpoint handlers, validation code, authorization logic, or audit infrastructure. Backend logic is expressed as runners — small programs following a three-phase read-act-write pattern through a shared library suite that handles all infrastructure concerns. A closed constraint vocabulary of nine types, three modifiers, six constraints, sixteen operations, and ten gate steps means security derives from structural limitation and the marginal cost of new application behavior approaches zero.

This paper enumerates 47 construction methods for building applications on OpsDB, covering domain analysis, schema design, policy configuration, runner implementation, frontend integration, and AI-assisted development. The methods apply across a spectrum from personal data platforms to enterprise backends under regulatory oversight, composing according to architecture position: primary backend for governed-state-dominant ap-

plications, split backend alongside specialized hot-path systems, or operational wrapper managing configuration and compliance around processing-dominant systems.

1. Development Lifecycle

Building an application on OpsDB follows a sequence that differs from conventional development at every stage. The sequence is: domain analysis, schema design, loader run, API verification, runner design, runner implementation, frontend integration. Each stage produces artifacts that the next stage consumes. No stage involves writing validation code, authorization logic, versioning systems, audit infrastructure, or API endpoint handlers.

1.1 The Artifact Set A complete AppDB application consists of four artifact types.

Schema YAML files. These declare every entity type, field, constraint, relationship, and governance configuration in the application. The schema is the source of truth for what data exists and what rules govern it. A typical governed-state-dominant application has 20–80 entity YAML files totaling 2–4K lines.

Runner specifications. These are OpsDB entities with typed JSON payloads that declare what each runner does, what it can access, what bounds it operates within, and what report keys it can write. Runner specs are governed data — versioned, change-managed, and auditable.

Runner code. Small programs — typically 150–300 lines each — that follow the get-act-set pattern using the shared library suite. A typical application has 3–8 runners. Each handles one domain-specific concern: pulling external data, reconciling state, computing derived quantities, dispatching notifications, generating reports, or producing AI-derived observations.

Frontend code. A thin consumer of the OpsDB API. It authenticates users via SSO, translates user actions into API calls, and renders responses. It has no database, no validation logic, no authorization logic. It renders what the API returns, including field-level omissions when access classification hides sensitive data. The frontend is conventional web or mobile development — React, Swift, Kotlin, server-rendered templates — and is not covered in depth here because OpsDB does not change how frontends are built, only what they connect to.

1.2 The Development Sequence **Step 1: Domain analysis.** Decompose the domain into entities, relationships, lifecycles, policies, schedules, external integrations, and hot-path processing (if any). The output is a domain map — a structured inventory of everything the application manages. This step is intellectual work, not code. It uses the comprehensive slicing method from the ops book: subdivide the domain preserving the whole, no information loss.

Step 2: Architecture position. Determine where OpsDB sits. Most business applications are governed-state-dominant — OpsDB is the entire backend. Applications with specialized processing needs identify the hot path and plan the runner bridge pattern. The taxonomy’s application suitability matrix provides the reference.

Step 3: Schema design. Translate the domain map into YAML files. Each entity type gets a file. Fields, types, constraints, relationships, governance fields, and discriminator patterns are declared. The schema conventions — singular names, lowercase underscores, hierarchical prefixes, FK naming, datetime suffixes, boolean prefixes — apply uniformly. This step typically takes hours to days, not weeks.

Step 4: Loader run. Run the schema loader against the YAML files. The loader parses, validates structural correctness, checks naming conventions, verifies foreign key targets exist, generates database DDL, creates tables with all reserved fields, and populates schema metadata tables. If the loader rejects the schema, the error identifies the specific violation. Fix the YAML file and re-run. When the loader succeeds, the API serves every declared entity with full properties — validation, authorization, versioning, change management, audit, search, retention.

Step 5: API verification. Write seed data through the API. Create entities, search them, update them through change sets, verify version history, check that constraints reject invalid data, confirm that access classification hides the right fields. This is testing — but the testing is against data declarations, not against code you wrote. The gate pipeline is the test runner. Every constraint violation you trigger confirms the schema is correct.

Step 6: Runner design. For each piece of backend logic the application needs — external integration, derived computation, notification dispatch, reconciliation, evidence production, AI observation — design a runner. Identify inputs, outputs, gating mode, trigger, bounds, and idempotency semantics. Write the runner spec as an OpsDB entity.

Step 7: Runner implementation. Write each runner using the library suite. The get phase reads from OpsDB and external sources. The act phase performs the domain-specific computation or integration. The set phase writes results back through the API. The library suite handles authentication, retry, circuit breaking, logging, correlation ID propagation, and scope enforcement. The runner handles only the domain decision.

Step 8: Frontend integration. Connect the frontend to the OpsDB API. Implement SSO authentication. Map user actions to API operations — search for list views, get_entity for detail views, submit_change_set for mutations, get_entity_history for activity feeds. Render approval status for pending change sets. Handle draft mode for fluid editing use cases.

2. Domain Analysis Method

Domain analysis produces the input for schema design. The method is systematic: enumerate exhaustively, classify each element, and identify relationships before writing any YAML. This is comprehensive design — build from the whole, not from accumulated parts.

2.1 Entity Enumeration List every noun in the domain that has identity, attributes, and lifecycle. A project management domain: project, task, milestone, comment, label, attachment reference, team, member, role. A billing domain: customer, account, plan,

subscription, invoice, line item, payment, credit, usage record. A healthcare domain: patient, provider, encounter, diagnosis, medication, order, result, care plan, insurance.

For each entity, ask: does this thing have its own identity (it is something, not just a property of something else)? Does it have attributes beyond the reference to its parent? Can it exist independently or does it exist only in context of another entity? Things with independent identity are entities. Things that are properties of other entities are fields on those entities. Things that represent relationships between entities are bridge tables.

2.2 Field Enumeration For each entity, list every attribute. For each attribute, determine type (from the nine available: int, float, varchar, text, boolean, datetime, date, json, enum), constraints (numeric ranges, string lengths, enum value sets, foreign key references), and modifiers (nullable, default, unique).

The closed vocabulary constrains this step productively. You cannot declare a regex constraint, so you do not need to design one. You cannot declare a conditional constraint, so cross-field invariants go to policy rows. You cannot declare computed defaults, so derived values go to runners. The vocabulary forces clean separation between what the schema handles (structural validation) and what runners handle (domain computation).

2.3 Relationship Classification Relationships between entities fall into five patterns.

Direct ownership. A task belongs to a project. The task has a `project_id` foreign key. The relationship is mandatory (not nullable) and hierarchical — deleting the project affects its tasks.

Optional reference. A task may reference an assignee. The task has a `assignee_user_id` foreign key that is nullable. The relationship is informational, not structural.

Many-to-many bridge. A task can have multiple labels. A label can apply to multiple tasks. A `task_label` bridge table has `task_id` and `label_id` foreign keys, both mandatory. The bridge table may have its own fields — `applied_time`, `applied_by_user_id`.

Polymorphic bridge. A comment can be attached to a task, a project, or a milestone. One bridge table per source-target pair: `task_comment`, `project_comment`, `milestone_comment`. Each bridge has the comment FK and the target FK. This preserves foreign key integrity — no polymorphic FK columns that could reference multiple tables.

Self-referential hierarchy. A task can have subtasks. The task has a nullable `parent_task_id` foreign key referencing its own table. Hierarchy traversal uses `get_dependencies` with the parent chain join path.

2.4 Lifecycle Identification Some entities progress through states with rules about valid transitions. Identify which entities have lifecycles and enumerate the states and transitions.

A task: draft → open → in_progress → review → done → archived. Valid transitions: draft→open, open→in_progress, in_progress→review, review→done, review→in_progress (returned for rework), done→archived, any→draft (reset). Invalid transitions: done→in_progress (must go through draft), archived→anything (terminal).

A subscription: trial → active → past_due → suspended → canceled → expired. Each transition has rules: trial→active requires payment method, active→past_due triggered by failed payment, past_due→suspended after configurable grace period, canceled is human-initiated, expired is system-triggered after cancellation period.

The states become an enum field on the entity. The valid transitions become policy rows evaluated at gate step 5. The transition rules that involve computation — “grace period elapsed,” “payment method valid” — become runner logic that proposes change sets when conditions are met.

2.5 Policy Enumeration For each entity and relationship, identify the governance requirements.

Approval rules. Which changes require human approval? Typically: changes to financial data, changes to access control configuration, changes to compliance-relevant fields, changes proposed by automated systems operating above a risk threshold. Low-risk changes — task status updates, comment additions, label assignments — auto-approve.

Access control. Which entities are visible to which roles? Which fields within an entity are sensitive? Patient diagnosis is visible only to the care team. Salary data is visible only to HR and the employee. Financial totals are visible only to billing administrators. These map to `_requires_group` (entity-level) and `_access_classification` (field-level) in the schema.

Retention. How long does version history persist for each entity type? Compliance-relevant entities may require 7-year retention. Operational telemetry may require 30-day retention. Personal notes may have no retention policy — keep forever on personal hardware.

Change management. Emergency review windows, segregation of duties requirements, mandatory review periods. These are policy rows, not schema declarations.

2.6 Schedule Enumeration Identify everything that recurs, expires, or has deadlines.

Recurring operations. Credential rotation every 90 days. Compliance audit quarterly. Invoice generation monthly. Report generation weekly. Backup verification daily. Each becomes a schedule entity with a typed payload — cron expression, rate-based, or calendar-anchored — connected to a runner spec.

Deadlines. Task due dates. Contract renewal dates. Certificate expiration dates. Each is a datetime field on the entity. A verifier runner reads approaching deadlines and produces evidence records or triggers notifications.

Event-triggered schedules. “Run compliance scan 24 hours after any policy change.” The schedule entity uses the event-triggered payload type, referencing the entity type and

field change that triggers it.

2.7 External Integration Enumeration Identify every external system the application interacts with.

Authorities. External systems that are the source of truth for some data. A payment processor is the authority for payment status. An identity provider is the authority for user identity. A cloud provider is the authority for resource state. Each authority gets an authority entity in OpsDB with a typed JSON payload describing the connection parameters.

Puller targets. For each authority, what data does the application need from it? Payment status, user groups, resource inventory. Each becomes a puller runner that reads from the authority and writes to observation cache tables.

Push targets. External systems that consume configuration from the application. An API gateway that reads route definitions. A CDN that reads content configuration. A notification service that receives dispatch requests. Each becomes a configuration push runner.

2.8 Hot Path Identification Determine whether the application has processing that cannot go through the gate pipeline.

The gate pipeline adds latency — authentication, five-layer authorization, schema validation, bound validation, policy evaluation, versioning, change management routing, audit logging, execution, response construction. For governed-state operations at human pace, this latency is invisible. For operations requiring sub-millisecond response, high-frequency writes, or compute-intensive processing, the pipeline is too heavy.

If the application has such processing, identify it, bound it, and plan the runner bridge. The hot path gets its own system. OpsDB governs everything around it. Runners push configuration to the hot path system and pull results back. The hot path system operates independently of OpsDB availability.

Most applications have no hot path. The gate pipeline handles human-pace CRUD, search, and change management without perceptible latency. The developer should not assume a hot path exists — verify through measurement, not assumption.

3. Schema Construction Methods

Schema construction translates the domain analysis into YAML files that the loader consumes. Each method addresses a specific schema design pattern.

3.1 Entity Design An entity YAML file declares the entity's name, fields, relationships, governance configuration, and notes. The minimal structure:

```
name: task
domain: project_management
versioned: true
notes: |
    A unit of work within a project. Lifecycle managed through
    status enum with transitions enforced by policy rows.
fields:
    - name: title
type: varchar
length: 255
nullable: false
    - name: description
type: text
nullable: true
    - name: status
type: enum
values: [draft, open, in_progress, review, done, archived]
nullable: false
default: draft
    - name: priority
type: int
min: 1
max: 5
```

```
nullable: false
default: 3
  - name: due_date
type: date
nullable: true
  - name: estimated_hours
type: float
min: 0.0
max: 10000.0
nullable: true
  - name: project_id
type: foreign_key
references: project
nullable: false
  - name: assignee_user_id
type: foreign_key
references: ops_user
nullable: true
  - name: _requires_group
type: foreign_key
references: ops_group
nullable: true
  - name: _access_classification
```



```
type: foreign_key

references: data_classification

nullable: true
```

Every field has an explicit type from the nine available. Every numeric field has explicit bounds. Every varchar has an explicit length. Every enum has an explicit value set. Every foreign key references a specific target table. The loader rejects anything outside this vocabulary.

The reserved fields — `id`, `created_time`, `updated_time`, `is_active`, `created_by_user_id`, `updated_by_user_id`, `version_serial` — are injected by the loader automatically. The developer does not declare them. The versioning sibling table (`task_version`) is generated automatically when `versioned: true` is set.

3.2 Type Selection The nine types serve specific purposes.

int — whole numbers with declared bounds. Use for: counts, quantities, priorities, ratings, sequence numbers, years. Always declare min and max. Prefer int over float for anything that should not have fractional values — currency in minor units (cents, not dollars), quantities of discrete items, scores on fixed scales.

float — decimal numbers with declared bounds. Use for: measurements, percentages, coordinates, rates. Always declare min and max. Be aware of floating-point representation — for financial calculations where precision matters absolutely, use int in minor units and let runners handle display formatting.

varchar — bounded-length strings. Use for: names, titles, identifiers, short descriptions, codes, slugs. Always declare length. The length is a real constraint — the database enforces it. Choose lengths deliberately: 63 for identifiers (DNS-compatible), 255 for titles and names, 1024 for short descriptions or URLs.

text — unbounded strings. Use for: long-form content, descriptions, notes, markdown content, prose. No length constraint. Use sparingly — most string fields have a natural upper bound and should be varchar.

boolean — true or false. Use with the naming conventions: `is_` prefix for present state (`is_active`, `is_published`, `is_billable`), `was_` prefix for past events (`was_approved`, `was_escalated`, `was_migrated`).

datetime — timestamp with timezone. Use for: event times, deadlines, schedule times. Always suffix with `_time` (`created_time`, `approved_time`, `expires_time`). Stored in UTC. Timezone handling is a runner concern, not a schema concern.

date — calendar date without time. Use for: due dates, birth dates, effective dates, billing cycle dates. Always suffix with `_date`. Use date instead of datetime when the time component is meaningless — a task due date is a date, not a datetime.

json — structured JSON with discriminator-based validation. Use for: typed payloads where different subtypes have different field sets (runner specs, schedule payloads, monitor configurations, evidence records). Always pair with a discriminator enum field — the `*_type` field determines which JSON schema validates the `*_data_json` field. Do not use json as a catch-all for unstructured data unless building the graduated formalization pattern (generic collection → proper schema).

enum — constrained value set. Use for: status fields, type discriminators, classification levels, category tags. Declare all values explicitly. Remember: enum values can be added but never removed (schema evolution rule). Plan the initial value set to cover known cases and leave conceptual room for additions.

foreign_key — reference to another entity. Use for: ownership, membership, reference, hierarchy. Always name as `referenced_table_id` or `role_referenced_table_id` when multiple FKs reference the same target.

3.3 Constraint Selection Six constraint mechanisms are available.

Numeric range (min/max on int and float). Enforced on every write. Choose bounds that reflect domain reality — priority 1–5, percentage 0.0–100.0, age 0–200, price 0–999999999 (in minor units). Bounds can be widened but never narrowed (evolution rule), so start with bounds that are correct, not bounds that are generous.

String length (length on varchar). Enforced on every write. Choose lengths that match the domain — 63 for machine identifiers, 255 for human-readable names, 1024 for descriptions. Lengths can be widened but never narrowed.

Enum value set (values on enum). Enforced on every write. Values can be added but never removed. The existing value “pending” exists forever even if deprecated. Plan initial sets carefully.

Foreign key reference (references on foreign_key). Enforced on every write — the referenced row must exist. This is relational integrity maintained by the database engine, not application code.

Nullable (nullable modifier). When false, the field must have a value on every write. When true, the field may be null. Default is false — fields are required unless explicitly marked nullable. This is a deliberate default: most fields should have values, and nullable fields should be the exception that the developer opts into.

Unique (unique modifier). When true, no two active rows in the table may have the same value for this field. Uniqueness applies to the active population — soft-deleted rows (`is_active=false`) do not participate in uniqueness checks. Uniqueness can be added to a field on an empty table but cannot be added to a field with existing duplicates, and cannot be tightened on a populated field (evolution rule).

3.4 The Discriminator Pattern When entities of the same structural type have different typed payloads — different kinds of monitors, different kinds of schedules, different kinds of evidence records — the discriminator pattern applies.

```

name: monitor

domain: monitoring

versioned: true

fields:
  - name: name
type: varchar
length: 255
nullable: false
  - name: monitor_type
type: enum
values: [prometheus_query, http_probe, tcp_probe, script_local,
        script_remote, k8s_event_watch, cloud_metric]
nullable: false
  - name: monitor_data_json
type: json
nullable: false
  - name: is_enabled
type: boolean
nullable: false

default: true

```

The `monitor_type` enum selects which JSON schema validates `monitor_data_json`. Each JSON schema is a separate YAML file under `schema/json_schemas/monitor/`. The loader registers the mapping. The gate pipeline validates the JSON payload against the schema selected by the discriminator value on every write.

This pattern replaces inheritance. There is no base class with subclasses. There is one table with a discriminator field and a validated JSON payload. All monitors share the same structural fields (name, type, is_enabled). Each monitor kind has its own validated payload shape. Adding a new monitor kind means adding an enum value and a JSON schema file — no code change.

3.5 Bridge Table Design Bridge tables handle many-to-many and polymorphic relationships with clean foreign key integrity.

Many-to-many bridge:

```
name: task_label

domain: project_management

versioned: false

fields:
  - name: task_id
type: foreign_key
references: task
nullable: false

  - name: label_id
type: foreign_key
references: label
nullable: false

  - name: applied_by_user_id
type: foreign_key
references: ops_user
nullable: false
```

The bridge table has no identity beyond its foreign keys. It exists to relate two entities. It may carry additional fields — who applied the label, when, why — that describe the relationship, not the related entities.

Polymorphic bridge:

When an entity (comment) can be attached to multiple different entity types (task, project, milestone), create one bridge table per target type:

```
name: task_comment
domain: project_management
versioned: false
fields:
  - name: task_id
type: foreign_key
references: task
nullable: false
  - name: comment_id
type: foreign_key
references: comment
nullable: false

name: project_comment
domain: project_management
versioned: false
fields:
  - name: project_id
type: foreign_key
references: project
nullable: false
```

```

    - name: comment_id

type: foreign_key

references: comment

nullable: false

```

This is more tables than a single polymorphic FK column, but every foreign key is a real foreign key to a real table, checked by the database on every write. No type column selecting which table the FK points to. No orphaned references. No application-level integrity checking. The database enforces it mechanically.

3.6 Hierarchy Design Self-referential foreign keys model hierarchies — organizational units, task/subtask trees, location trees, category trees.

```

name: category

domain: catalog

versioned: true

fields:

    - name: name

type: varchar

length: 255

nullable: false

    - name: parent_category_id

type: foreign_key

references: category

nullable: true

    - name: depth

type: int

min: 0

```

```
max: 20
```

```
nullable: false
```

```
default: 0
```

The `parent_category_id` is nullable — root categories have no parent. The `depth` field is maintained by a runner that traverses the parent chain and sets depth on create or reparent. The depth bound (max: 20) prevents unbounded hierarchy depth.

Traversal uses `get_dependencies` with the parent chain join path. The search API can filter by parent, by depth, or by subtree membership. A runner that computes hierarchy metadata — full path string, child count, leaf status — writes these as fields on the entity or as observation cache rows, depending on whether the computed values are governed state or derived cache.

3.7 Governance Field Configuration Three governance fields control per-entity access beyond role-based authorization.

`**_requires_group**` scopes entity visibility to members of a specific group. When set, only members of the referenced group (or administrators) can read or write the entity. Use for: project-scoped data (only project members see project tasks), team-scoped data (only team members see team resources), personal data (only the owner sees their private items).

```
- name: _requires_group
```

```
type: foreign_key
```

```
references: ops_group
```

```
nullable: true
```

When null, the entity is visible to any authenticated user with appropriate role permissions. When set, group membership is checked at gate step 2 layer 2.

`**_access_classification**` controls field-level visibility. When a field has an access classification, only users with roles that include that classification level can see the field value. Use for: salary data (restricted to HR), medical details (restricted to care team), financial details (restricted to billing administrators).

The classification is set per entity row, not per field definition. This means different rows of the same entity type can have different classification levels. A patient record for a VIP might have higher classification than a standard record.

`versioned: true` opts the entity into change management. Every write produces a version row. Change sets route through approval. Version history is queryable. This is the default

for entities where change history matters. Set to false for ephemeral or high-frequency entities where versioning would create noise — observation cache tables, job records, temporary coordination state.

3.8 Draft Mode Configuration For entities where interactive editing is the primary use pattern, draft mode governance flags reduce friction without eliminating structural safety.

```
name: document
domain: knowledge_management
versioned: true
draft_mode:
  _autoversion_disabled: true
  _edit_latest_version: true
  _audit_logs_disabled: true
notes: |
  Draft mode enabled. Interim editing states are not individually
  versioned or audited. Version commits engage full governance.
  Properties weakened: per-write versioning, per-write audit.
  Properties preserved: schema validation, authorization,
  committed-version history, committed-version audit.
  Acceptable because: document content is iterative drafting
  where per-keystroke versioning creates noise without value.
```

The property analysis in the notes field is not optional guidance — it is the schema steward’s verification that the tradeoff is understood. Before setting draft mode flags, the developer must enumerate which properties are weakened and confirm that the weakened properties are not required for the entity’s role in the application.

Auth and authorization always run. Schema validation always runs. Bound validation always runs. Draft mode never weakens security or structural integrity. It weakens recording — versioning, audit, change management — for interim saves between explicit com-

mits.

3.9 Schema Organization Schema files are organized in a directory structure that mirrors the domain taxonomy.

```
schema/

  directory.yaml          # imports all domain directories
  conventions/

  reserved.yaml          # reserved field definitions
  domains/

  01_identity/           # users, groups, roles
  02_core/               # primary domain entities
  03_workflow/           # lifecycle and process entities
  04_policy/             # policies, rules, classifications
  05_schedule/           # schedules, deadlines
  06_integration/        # authority pointers, external refs
  07_observation/        # observation cache tables
  08_audit/              # audit, evidence, compliance
  09_runner/             # runner specs, jobs, capabilities
  json_schemas/

  monitor/              # per-discriminator JSON schemas
  schedule/

  evidence_record/

  runner_spec/

  meta/
```

```
_schema_meta.yaml          # schema metadata entity types
```

The directory numbering creates a natural dependency order — identity entities exist before core entities reference them, core entities exist before policies reference them. The `directory.yaml` file imports domain directories in order. Entity files are leaf-level — they do not import other files.

4. Policy Construction Methods

Policies govern application behavior through data rows evaluated by the gate pipeline. Every policy is itself an OpsDB entity — versioned, change-managed, and auditable. Changing application behavior means submitting a change set that modifies policy rows, not deploying new code.

4.1 Approval Rule Design Approval rules determine which changes require human approval and from whom. An approval rule is a policy row that matches against change set characteristics and declares the approval requirements.

The matching criteria include: entity type (which tables are affected), field names (which specific fields are changing), namespace (which organizational scope), data classification level (how sensitive the data is), security zone (which environment), and the proposer's role (different rules for human vs automated proposers).

Design approval rules from the stakes, not from the entity structure. Ask: if this change is wrong, what is the impact? A task status change to “done” has low impact — auto-approve. A project budget change has financial impact — require the project owner's approval. A data classification change affects who can see what — require the data steward's approval. A production policy change affects system behavior — require two approvers from different roles (segregation of duties).

The calibration for AI proposers follows the same logic. An AI runner that proposes changes to low-sensitivity observation summaries can be auto-approved. An AI runner that proposes changes to financial records based on reconciliation results requires human review. The policy rows that govern AI-proposed changes are the same mechanism as all other approval rules — they just match on the proposer's service account identity.

```
# Approval rule: budget changes require project owner

name: budget_change_approval

policy_type: approval_rule

policy_data_json:

  match:
```

```

entity_types: [project]

field_names: [budget_amount, budget_currency]

    require:

approver_count: 1

approver_source: ownership_bridge

approver_role: project_owner

# Approval rule: AI-proposed entity changes require human review

name: ai_proposal_review

policy_type: approval_rule

policy_data_json:

    match:

proposer_type: service_account

proposer_role: ai_runner

    require:

approver_count: 1

approver_source: entity_owner

```

4.2 Access Control Design Access control uses three complementary mechanisms.

Role-based access (layer 1). Define roles that map to job functions: admin, manager, member, viewer, billing_admin, hr_admin. Assign permissions per role: which entity types can be read, which can be written. This is the coarsest control — it determines what categories of data a role can interact with.

Entity-level scoping (layer 2). Set `_requires_group` on entities to restrict visibility to group members. A project’s tasks are visible only to project members by setting `_requires_group` to the project’s member group. This scoping is per-row — different projects have different groups, so different teams see different projects.

Field-level classification (layer 3). Set `_access_classification` on entity rows to control which fields are visible. A patient record with classification “restricted”

hides diagnosis and treatment fields from roles that lack the restricted access grant. The API response includes the field names with a metadata flag indicating the value is hidden, so the frontend can render “access restricted” rather than an empty field.

Design access control from the data sensitivity outward, not from the user roles inward. Ask: what data exists, how sensitive is each element, who legitimately needs to see it? Then create the roles and groups that express those access needs. This prevents the common failure mode of designing roles first and discovering later that the role structure doesn’t match the data sensitivity boundaries.

4.3 State Transition Rules For entities with lifecycle state machines, transition rules are policy rows that the gate pipeline evaluates at step 5.

```
name: task_transitions
policy_type: state_transition
policy_data_json:
  entity_type: task
  field: status
  transitions:
- from: draft
  to: [open]
- from: open
  to: [in_progress, draft]
- from: in_progress
  to: [review, draft]
- from: review
  to: [done, in_progress]
- from: done
  to: [archived]
```

```
- from: archived

  to: []      # terminal state
```

When a change set proposes a status field change on a task, the gate pipeline checks the transition against this policy. If the transition is not in the declared set, the change set is rejected. The rejection is audited — the audit log records the attempted invalid transition, who proposed it, and when.

Transition rules can include conditions evaluated by runners. “A task can only move to done if all subtasks are done” is not expressible as a static policy row — it requires querying subtask state. A verifier runner handles this: before a task can transition to done, the runner checks subtask completion and writes an evidence record. The approval rule for the done transition can require this evidence as a precondition.

4.4 Retention Policy Design Retention policies determine how long versioned data, observation cache entries, and audit records persist.

```
name: task_retention

policy_type: retention

policy_data_json:

  entity_type: task

  version_history_days: 365

  observation_cache_days: 30

  audit_log_days: 2555      # 7 years for compliance
```

A reaper runner reads retention policies and enforces them — soft-deleting version rows past their retention horizon, removing expired observation cache entries, preserving audit entries for their declared retention period.

Design retention from regulatory requirements first, operational needs second, storage costs third. Compliance-relevant entities may have retention periods set by law (7 years for financial records, varies by jurisdiction for medical records). Operational entities use retention periods that match their operational relevance — 30 days for observation cache, 90 days for runner job records. Storage cost should never drive retention for compliance-relevant data.

4.5 Cross-Field Invariant Rules The schema vocabulary does not include conditional constraints. Cross-field invariants — “if status is active then start_date must be set,” “if payment_method is credit_card then card_token must be present” — are policy rows evaluated at gate step 5.

```

name: active_requires_start_date

policy_type: semantic_invariant

policy_data_json:

  entity_type: task

  condition:

field: status

equals: active

  require:

field: start_date

not_null: true

  message: "Active tasks must have a start date"

```

This separation keeps the schema vocabulary simple and the validation logic inspectable. Every cross-field rule is a queryable policy row. You can enumerate all invariants for an entity type with a single search query. Changing an invariant is a change set, audited and reversible. In conventional development, cross-field validation is buried in application code — scattered across model validators, service methods, and middleware — and discovering all the rules requires reading the codebase.

5. Runner Construction Methods

Runners implement the backend logic that declarations cannot express. Each runner follows the get-act-set pattern, uses the shared library suite for infrastructure concerns, and operates within declared bounds.

5.1 Runner Design Method Before writing code, design the runner by answering seven questions.

What does it read? List every OpsDB entity type and external source the runner consults. A notification runner reads: change set entities (to detect new approvals needed), schedule entities (to detect approaching deadlines), on-call assignment entities (to resolve recipients), authority entities (to get notification channel configuration).

What does it write? List every OpsDB table and external system the runner modifies. The notification runner writes: observation cache entries (delivery receipts), and

dispatches to external notification channels (email, webhook, chat) via the library suite.

What is the gating mode? Direct write for observation-like output that bypasses change sets but is still authenticated, validated, and audited. Auto-approve for low-risk changes that create change sets but transition through approval automatically. Approval-required for high-risk changes that route to human reviewers.

What triggers execution? Scheduled (cron or rate-based — runs every N minutes), event-driven (triggered by a state change the runner watches for), or long-running (continuous loop with sleep between cycles).

What are the bounds? Retry budget (how many times to retry a failed action), execution time limit (maximum seconds per cycle), scope per cycle (maximum entities to process per run), memory bounds (maximum heap allocation). These are declared in the runner spec and enforced by the library suite.

What does idempotency mean? If the runner runs twice on the same input, what happens? For a notification runner: sending the same notification twice is undesirable, so the runner tracks which notifications have been sent using observation cache rows keyed by change set ID and recipient. For a reconciler: applying the same correction twice produces the same end state because the change set is idempotent — if the value is already correct, no change is proposed.

What report keys can it write? The runner declares which observation cache tables and fields it can write to. The API and library suite both validate writes against these declarations. A notification runner can write to `observation_cache_state` with keys related to notification delivery. It cannot write to `observation_cache_metric` or to governed entity tables outside its declared scope.

5.2 Puller Runner Construction A puller runner imports data from an external system into OpsDB observation cache tables.

Structure:

get phase:

- read runner spec from OpsDB (connection params, schedule, bounds)
- read authority entity from OpsDB (endpoint URL, auth reference)
- read last sync cursor from observation cache (if incremental)

act phase:

- connect to external API via library suite
- paginate through external data

- transform each external record to OpsDB entity shape
- handle rate limits (library suite manages backoff)

set phase:

- write transformed records to observation cache via API
- update sync cursor in observation cache
- write sync summary (record count, errors, duration) to observation cache

Schema mapping. The transformation from external format to OpsDB entity shape is the puller’s primary domain logic. A Stripe payment puller reads Stripe’s payment intent objects and maps them to OpsDB observation cache rows: `stripe_payment_id` → observation key, `amount` → `observed_amount`, `status` → `observed_status`, `created` → `observed_time`. The mapping is explicit code in the runner — typically a function per external entity type, 20–40 lines each.

Incremental sync. Most external APIs support cursor-based pagination or timestamp-based filtering. The puller stores its last sync cursor as an observation cache entry. Each cycle reads from the cursor forward. Full resync is triggered by deleting the cursor entry — the puller starts from the beginning on its next cycle. This is idempotent: running a full resync produces the same end state as incremental sync that never missed an event.

Freshness. Every observation cache entry written by a puller carries a freshness timestamp — the time the puller read the data from the external source. Consumers of the observation data can filter by freshness — “show me only observations from the last hour” — to avoid acting on stale data. The search API’s freshness annotation parameter makes this a standard query predicate.

5.3 Reconciler Runner Construction A reconciler compares desired state (from governed entities) with observed state (from observation cache or external queries) and proposes or applies corrections.

Structure:

get phase:

- read desired state from governed entities via search API
- read observed state from observation cache via search API
- (or read observed state directly from external system via library suite)

act phase:

- for each desired entity, find the corresponding observed state
- compute the diff: fields where desired $\neq$ observed
- for each diff, determine the correction action
- filter out diffs within acceptable tolerance

set phase:

- for each correction:

- if auto-approve: submit change set with correction, auto-approved by policy
- if approval-required: submit change set with correction, routes to approvers
- if direct external action: execute via library suite, write result as observed

Convergence behavior. The reconciler is level-triggered, not edge-triggered. It does not react to events (“this value changed”). It compares current desired state with current observed state on every cycle. If an event is missed — a webhook fails, a notification drops — the next reconciler cycle detects the discrepancy from the state comparison. This is the R3 idempotency principle and R8 level-triggered-over-edge-triggered from the ops book applied to runner design.

Tolerance. Not every diff is worth correcting. A reconciler comparing expected inventory counts with warehouse-observed counts might tolerate a variance of ± 2 items before proposing a correction. Tolerance thresholds are configurable in the runner spec’s typed JSON payload — they are data, not hardcoded constants.

Partial drift. When the reconciler finds multiple diffs across multiple entities, it can propose a single bulk change set covering all corrections, or individual change sets per entity. Bulk change sets are atomic — all corrections apply or none do. Individual change sets allow partial progress — some corrections can be approved while others are pending review. The choice depends on whether the corrections are interdependent. Independent corrections (each entity’s count is independently correct or incorrect) use individual change sets. Interdependent corrections (a set of accounts must balance to zero) use bulk change sets.

5.4 Verifier Runner Construction A verifier checks that expected conditions hold and produces evidence records documenting the result.

Structure:

get phase:

- read schedules that are due for verification
- read target entities referenced by each schedule
- read prior evidence records (to detect regressions)

act phase:

- for each due schedule:
- evaluate the verification condition against the target
- determine pass or fail
- assemble supporting data (what was checked, what was found)

set phase:

- write evidence record per verification with:
- pass/fail result
- verification time
- supporting data as typed JSON payload
- reference to schedule and target entity
- update schedule's last_verified_time

Evidence record typed payloads. Each verification type has its own JSON schema for the evidence record payload. A certificate validity verifier writes: certificate subject, issuer, expiration date, days until expiration, chain validation result. A backup verification verifier writes: backup timestamp, backup size, restore test result, duration. The typed payload ensures every evidence record of a given type has consistent structure — queryable, comparable across time, and usable by compliance reporting.

Regression detection. The verifier reads prior evidence records for the same target. If the prior result was pass and the current result is fail, the verifier can trigger escalation — writing an alert entity, notifying the on-call team, or proposing a change set that flags the entity for remediation. Regression detection is domain logic in the runner, not infrastructure — the runner decides what constitutes a regression for its verification type.

5.5 Notification Runner Construction A notification runner detects state transitions that warrant notification and dispatches messages through configured channels.

Structure:

get phase:

- read recent change sets (since last notification cycle)
- read pending approval requests
- read approaching deadlines (schedules with due dates within threshold)
- read notification configuration (which transitions trigger which channels)
- read recipient resolution data (on-call assignments, ownership bridges)

act phase:

- for each trigger event:
 - resolve recipients from ownership/stakeholder bridges and on-call assignments
 - select channel(s) from notification configuration
 - format message using configured template
 - deduplicate against prior notifications (check observation cache)

set phase:

- dispatch via library suite (email, webhook, chat API)
- write delivery receipt to observation cache (keyed by event + recipient)
- write delivery failures to observation cache for retry on next cycle

Channel configuration as data. Notification channels are authority entities with typed payloads: email (SMTP configuration), webhook (URL, headers, auth), chat (Slack/Teams API configuration). Adding a notification channel means creating an authority entity through a change set. Removing a channel means deactivating the authority entity. The runner reads the active authority entities and dispatches through them. No code change for channel changes.

Recipient resolution. The runner does not maintain its own recipient lists. It resolves recipients from OpsDB data: ownership bridges (who owns the affected entity), stakeholder bridges (who has declared interest), on-call assignments (who is currently responsible), and escalation paths (who to contact if the primary recipient does not respond within a configured window). All of these are governed entities — changing who receives notifications means changing data, not code.

5.6 Configuration Push Runner Construction A configuration push runner reads governed state from OpsDB and pushes it to an external system in that system’s native format.

Structure:

get phase:

- read governed entities that the external system consumes
- read the external system's current configuration (via library suite)
- read the runner spec for transformation rules and connection params

act phase:

- transform governed entities into the external system's native format
- compute the diff between current external config and desired config
- generate the update payload

set phase:

- push the update to the external system via library suite
- write push result (success/failure, timestamp, diff summary) to observation
- on failure: log structured error, increment retry counter, respect retry bu

Transformation as domain logic. The transformation from OpsDB entity shape to external format is the runner’s primary contribution. An API gateway configuration push runner reads route definition entities, rate limit policy entities, and consumer quota entities from OpsDB, and produces the gateway’s native configuration format — a YAML file, a JSON API payload, a protobuf message. This transformation is 50–100 lines of mapping code. The library suite handles the connection, authentication, retry, and error handling for the push.

Diff-based push. The runner does not blindly push the full configuration on every cycle. It reads the external system’s current configuration, computes the diff against the desired configuration from OpsDB, and pushes only the changes. This minimizes disruption to the external system and makes each push operation smaller and faster. If the diff is empty — the external system already matches the governed state — the runner does nothing and writes a “no changes” observation.

Cache independence. The external system should cache the configuration it receives. If OpsDB is unavailable, the external system continues operating with its cached configura-

tion. When OpsDB returns, the runner catches up and pushes any accumulated changes. The external system never depends on OpsDB at runtime — only at configuration refresh time. This is the local cache plus global truth principle from the ops book: the external system has a local copy, OpsDB is the global truth.

5.7 Observation Pull Runner Construction An observation pull runner reads results from an external system and writes them to OpsDB observation cache tables.

This is the complement of the configuration push runner. Together they form the bridge pattern for split-backend and wrapper architectures: push configuration out, pull results back.

Structure:

get phase:

- read runner spec (external system connection, polling interval, scope)
- read authority entity (endpoint, auth reference)
- read last poll cursor from observation cache

act phase:

- connect to external system's results API via library suite
- read results since last poll cursor
- transform results to observation cache format

set phase:

- write results to observation cache with freshness timestamps
- update poll cursor
- write poll summary (record count, errors, latency) to observation cache

The pull runner handles the external system's eventual consistency. If the external system reports results with delay — a payment processor that settles transactions hours after authorization — the pull runner writes observations with the timestamp from the external system, not the pull time. Consumers querying the observation cache see when the event actually occurred, not when OpsDB learned about it.

5.8 AI Observation Runner Construction An AI observation runner uses an LLM to generate derived observations over governed data. The output is always observation

cache — never governed state.

Structure:

get phase:

- read runner spec (model, token budget, prompt template reference, regeneration trigger, target entity types)
 - read governed entities that form the context
 - traverse relationships to build complete context
- (entity → dependencies → change sets → alerts → observations)

act phase:

- assemble context into structured prompt
- call LLM via library suite (with retry, timeout, circuit breaking)
- validate response length against configured character budget
- validate response structure if structured output is expected

set phase:

- write AI observation to observation cache with:
- freshness timestamp
- character budget used
- model identifier
- source entity references (which entities were summarized)
- prompt hash (for reproducibility tracking)
- on LLM failure: write error observation, continue to next entity

Configuration via runner spec:

name: `incident_summary_runner`

```

runner_type: ai_observation

runner_data_json:

  model: "claude-sonnet"

  max_tokens: 500

  character_budget: 2000

  regeneration_trigger: "source_entity_change"

  target_entity_types: [incident]

  context_traversal:

    - incident -> affected_service (via bridge)

    - affected_service -> service_connection (dependencies)

    - incident -> change_set (recent, last 24h)

    - incident -> alert_fire (related alerts)

  prompt_template_ref: "incident_summary_v1"

  observation_cache_table: observation_cache_state

  observation_key_prefix: "ai_summary_incident_"

```

Multiple granularities. The same runner code serves different summary needs through different runner specs. An 80-character headline for a dashboard ticker has `character_budget: 80` and a terse prompt template. A 2000-character incident summary for the war room has `character_budget: 2000` and a detailed prompt template. A 12-character severity classification has `character_budget: 12` and a classification-focused prompt template. Three runner specs, one runner implementation.

Regeneration. The runner regenerates observations when source data changes. The `regeneration_trigger` field in the spec determines when: on any source entity change, on a schedule, or on demand. On-change regeneration keeps summaries fresh as incidents evolve. Scheduled regeneration limits LLM costs for stable data. On-demand regeneration lets operators request fresh summaries when needed.

Containment. If the LLM hallucinates, the governed data is unaffected. The AI observation is a cache row that will be overwritten on the next regeneration cycle. The freshness timestamp tells consumers how recent the summary is. The source entity references let consumers verify the summary against the actual data. The prompt hash enables repro-

ducibility investigation — given the same context and prompt, does the LLM produce the same hallucination?

5.9 Custom Domain Runners Some domain logic does not fit the standard runner kinds. A grade computation runner, an ELO rating runner, a budget tracking runner, an invoice generation runner — each performs domain-specific computation that reads governed state and writes results back.

The get-act-set pattern still applies. The library suite still handles infrastructure. The runner still declares its scope and bounds.

Example: invoice generation runner.

get phase:

- read active subscriptions with `billing_cycle_date ≤ today`
- read plan entities for each subscription (pricing data)
- read usage records since last invoice for metered plans
- read tax policy data for jurisdiction-based tax rates
- read credit balance for each customer account

act phase:

- for each billable subscription:
- compute base charge from plan pricing
- compute usage charges from usage records $\times$ metered rates
- compute proration for mid-cycle changes
- apply credits from credit balance
- compute tax from jurisdiction rules
- generate line items

set phase:

- submit change set creating invoice entity with line items
- submit change set updating credit balance if credits applied

- write invoice generation summary to observation cache

The runner is 200–300 lines. The pricing data, tax rules, proration logic, and credit application are domain computations — the runner’s actual contribution. Everything else — reading data, writing change sets, handling errors, retrying failures, logging, authentication — is library suite calls.

5.10 The Server Runner Model Runners operate as threads within a server runner process. The server runner is a single Go binary that hosts multiple runner threads, manages their lifecycle, and shares infrastructure resources.

Thread configuration. The server runner reads runner spec entities from OpsDB to determine which runners to instantiate and how many threads each gets. Configuration is data — changing which runners run, how many threads each gets, and what cycle timing each uses means changing runner spec rows through change sets.

```
# Runner thread configuration (in runner_spec entity)

name: notification_runner_spec

runner_type: notification

runner_data_json:

    thread_count: 2

    cycle_interval_seconds: 30

    retry_budget: 3

    execution_timeout_seconds: 60

    scope_per_cycle: 100
```

Shared resources. All runner threads share the database connection pool, the library suite instances (with per-runner scope enforcement), the logging infrastructure, and the metrics collection. This eliminates the per-runner operational overhead that separate processes would require — no per-runner deployment, no per-runner monitoring configuration, no per-runner resource allocation.

Scope isolation. Despite sharing a process, runner threads are isolated by scope. Each thread’s library suite instance is configured with the runner’s declared capabilities and targets. A notification runner thread cannot write to billing tables because its scope declaration does not include them. The library suite checks every API call against the thread’s scope before sending it. The API gate checks again on receipt. Double enforcement —

library-side and API-side — means a bug in one runner thread cannot affect data outside its declared scope.

Scaling. When the server runner needs more capacity, you run additional instances of the same server runner binary. Each instance reads the same runner specs and distributes work through the standard coordination mechanisms — advisory locks, runner job claims, and observation cache cursors. The 0/1/N rule applies: one server runner type, N instances for scaling.

Health monitoring. Each runner thread writes health observations to the observation cache on every cycle: cycle start time, cycle duration, records processed, errors encountered. A monitoring runner (or an external monitoring system pulling from the observation cache) detects unhealthy threads — prolonged cycle times, increasing error counts, stuck threads. The health observations are the same mechanism as any other observation — queryable through the search API, subject to freshness semantics, and visible to dashboards.

6. Frontend Integration Methods

The frontend is a thin consumer of the OpsDB API. It has no database. It implements no validation, authorization, versioning, or audit logic. It authenticates users, translates user actions into API calls, and renders responses.

6.1 Authentication Integration The frontend authenticates users through SSO delegation to an identity provider. The OpsDB API accepts tokens from the configured IdP(s). The frontend's responsibility is acquiring the token (through OAuth2 / OIDC flows) and passing it with each API request.

The frontend does not maintain user sessions beyond the token lifecycle. Session management is token management — refresh tokens, handle expiry, redirect to IdP for re-authentication. The OpsDB API validates the token on every request at gate step 1. No session state in the frontend's backend-for-frontend layer. No session database. No session synchronization across frontend instances.

6.2 Read Patterns **List views.** Call `search` with filter predicates, projection (which fields to return), ordering, and cursor-based pagination. The search API returns only entities the authenticated caller can access — multi-tenancy is invisible to the frontend code. Fields the caller cannot see due to access classification are omitted from the response with metadata indicating the omission.

```
POST /api/search

{

  "entity_type": "task",
```

```

    "filters": {
"and": [
    {"field": "project_id", "op": "eq", "value": "proj_123"},
    {"field": "status", "op": "in", "value": ["open", "in_progress"]},
    {"field": "assignee_user_id", "op": "eq", "value": "current_user"}
]
    },
    "fields": ["id", "title", "status", "priority", "due_date", "assignee_user_
"order_by": [{"field": "priority", "direction": "asc"}, {"field": "due_date
    "page_size": 25,
    "cursor": null
}

```

Detail views. Call `get_entity` with the entity type and ID. The response includes all fields the caller can see plus metadata: version serial, created/updated timestamps, created/updated by identity. If the entity is versioned, the current version is returned. Named join paths can include related entities in a single request — a task detail view can include the project, the assignee, and recent comments through declared join paths.

Activity feeds. Call `get_entity_history` to retrieve the version chain for an entity. Each version row includes the full state at that version, the change set that produced it, the proposer, the approver(s), and the timestamp. The frontend renders a timeline of changes with attribution. For a project-level activity feed, query change sets filtered by entity types within the project’s scope.

Dashboard and reporting. Dashboards are search queries with aggregation handled by the frontend or by observation cache tables populated by reporting runners. Simple counts and distributions can be computed client-side from search results. Complex aggregations — time-series trends, cross-entity correlations, statistical summaries — are computed by a reporting runner that writes results to observation cache tables, which the frontend reads through the same search API.

Point-in-time views. Call `get_entity_at_time` with a timestamp to reconstruct the state of any entity at any prior point. The response is the version row that was current at the specified time — a single row lookup, not a chain replay. This supports audit views (“what did this record look like on March 15th”), incident investigation (“what was the

configuration when the outage started”), and regulatory queries (“show the state of this account at end of fiscal year”).

6.3 Write Patterns **Standard writes.** The frontend submits a change set containing the proposed field changes and a reason string.

```
POST /api/change_set/submit

{
  "entity_type": "task",
  "entity_id": "task_456",
  "changes": [
    {"field": "status", "old_value": "open", "new_value": "in_progress"},
    {"field": "assignee_user_id", "old_value": null, "new_value": "user_789"}
  ],
  "reason": "Starting work on authentication redesign task"
}
```

The `old_value` fields enable optimistic concurrency — if the entity has changed since the frontend read it, the submit fails with a conflict response. The frontend can then re-read, merge if appropriate, and re-submit.

Auto-approved writes. For changes matching auto-approve policy rules, the change set transitions through `draft` → `submitted` → `approved` → `applied` within the same request cycle. The frontend experiences this as an immediate write. But the change set is created, the version row is written, the audit entry is logged, and the change is reversible through a subsequent change set.

Approval-required writes. For changes matching approval-required policy rules, the change set transitions to `pending_approval`. The API response includes the required approvals — who needs to approve, from what roles, how many approvals are needed. The frontend renders a pending state: “Your change to project budget is awaiting approval from the project owner.”

The frontend for approvers shows pending change sets with the proposed changes, the reason, the proposer’s identity, and the approval/rejection controls. Approving or rejecting is itself an API call — `approve_change_set` or `reject_change_set` with a reason. When all required approvals are received, the change set executor runner applies the changes.

Entity creation. Creating a new entity is a change set where all fields are `new_value` with no `old_value`. The change set creates the entity, the version row, and the audit entry atomically.

Bulk operations. The `bulk_submit` operation accepts multiple entity changes in a single change set. The changes are applied atomically — all succeed or all fail. Use for operations that must be consistent across entities: creating an invoice with line items, assigning multiple tasks to a sprint, applying a configuration change across multiple entities.

Emergency path. For break-glass situations, the `emergency_apply` operation bypasses normal approval routing. It requires the caller to have the emergency role. It applies the change immediately with reduced approvals. It sets a mandatory flag in the audit log. It creates a review schedule entry requiring post-incident review within a configurable window. The frontend for emergency operations should be distinct from the normal editing interface — a separate UI path that makes the extraordinary nature of the action visible.

6.4 Draft Mode Integration For entities with draft mode governance flags enabled, the frontend implements continuous autosave with explicit version commits.

Autosave. On every user edit (debounced — typically 1–3 seconds after the last keystroke), the frontend sends a direct write to the API. Auth, schema validation, and bound validation run. Versioning, change management, and audit logging are skipped per the draft mode flags. The current row is updated in place. If the browser crashes, the user loses at most a few seconds of work.

Version commit. When the user explicitly saves (a deliberate action — a button click, a keyboard shortcut, not an autosave), the frontend sends a version commit request. All ten gate steps run. A version row is created with the full state. A change set records the delta from the prior committed version. The audit log records the commit event. The commit becomes part of the permanent version history.

Rendering draft vs committed state. The frontend distinguishes between the current working state (the entity’s current row, which may include uncommitted edits) and the committed version history (the version rows). An editing interface shows the current working state with a “save version” button. A history interface shows committed versions with diffs between them. The working state is ephemeral — it represents the user’s current draft. The committed versions are permanent — they represent intentional checkpoints.

Undo/redo. Undo within a draft session is a frontend concern — standard browser undo or application-level undo stack. Undo across committed versions is a governed operation — the frontend submits a change set that restores the prior version’s field values. This change set goes through full governance: validation, authorization, approval (if required), versioning, and audit. Rolling back to a prior version is recorded as a forward change, not a deletion of history.

6.5 Multi-Tenancy in the Frontend The frontend does not implement multi-tenancy. It passes the authenticated user’s identity with every API call. The API returns only what

the caller can access.

Search results are pre-filtered. A search for tasks returns only tasks the caller can see — filtered by role permissions (layer 1) and group membership (layer 2). The frontend does not need to add tenant filters to queries. The API handles it.

Field-level omissions. When access classification hides fields, the API response includes the field name with a metadata flag: `{"field": "salary", "access": "restricted", "value": null}`. The frontend renders this appropriately — “Access restricted” rather than an empty field, so the user knows data exists but is not visible to them.

Group-scoped navigation. The frontend can query the user’s group memberships and render navigation accordingly. A user who is a member of three project groups sees three projects. Adding the user to a fourth group (a change set on the group membership entity) immediately makes the fourth project visible — no frontend deployment, no configuration change, no cache invalidation.

7. Application Patterns by Type

Each architecture position has a distinct construction pattern. The patterns below describe what to build, what OpsDB provides, and where the development effort concentrates.

7.1 Governed-State-Dominant Pattern **Architecture position:** OpsDB is the primary backend. No hot path. 95%+ of the application is governed state management.

What you build: Schema YAML files for all domain entities. Policy rows for approval rules, access control, retention, and lifecycle rules. 3–6 runners for notification, integration, reporting, and domain computation. A frontend that consumes the API.

What OpsDB provides: The entire API layer — CRUD, search, change management, versioning, audit. Authentication and five-layer authorization. Schema validation and bound validation. Optimistic concurrency. Point-in-time state reconstruction. Retention enforcement.

Where effort concentrates: Domain analysis and schema design (60% of backend effort). Runner implementation (30%). Policy configuration (10%). The frontend is a separate effort of conventional size — OpsDB does not reduce frontend work, only backend work.

Representative applications: Project management, CRM, HR platform, compliance platform, inventory management, procurement, legal case management, education platform, personal data platform.

Construction sequence:

1. Domain analysis: enumerate entities, relationships, lifecycles, policies
2. Schema design: write YAML files for all entities
3. Loader run: generate database structure

4. Seed data: create initial policy rows, approval rules, role definitions, group memberships
5. API verification: create test entities, verify constraints, test approval flows
6. Runner implementation: build domain runners (notification, integration, computation)
7. Frontend development: build UI consuming the API
8. Policy tuning: adjust approval rules, access control, and retention as real usage reveals needs

7.2 Split-Backend Pattern **Architecture position:** OpsDB governs most state. A specialized system handles a bounded hot path. Runners bridge the two. 70–90% governed state.

What you build: Schema YAML for the governed portion (which is most of the data model). The hot-path service as a conventional system optimized for its specific requirement. Configuration push runners that format governed state for the hot-path service. Observation pull runners that bring results back. A frontend that consumes both the OpsDB API (for governed operations) and the hot-path service API (for real-time operations).

What OpsDB provides: Everything from the governed-state pattern, plus the runner infrastructure for bridging. The hot-path service gets its configuration from OpsDB via runners and operates independently.

Where effort concentrates: The hot-path service (40–50% of total backend effort). Schema design for the governed portion (20–25%). Runner bridges (15–20%). Policy configuration (5–10%).

Representative applications: E-commerce (cart/checkout hot path), booking systems (reservation contention hot path), subscription billing (payment processing hot path), IoT fleet management (telemetry ingestion hot path), content management (content delivery hot path).

Construction sequence:

1. Domain analysis: identify governed entities AND hot-path requirements
2. Schema design for governed portion
3. Hot-path service design and implementation
4. Runner bridge design: what configuration flows out, what results flow back
5. Configuration push runner implementation
6. Observation pull runner implementation
7. Loader run and API verification for governed portion
8. Integration testing: verify the bridge — config changes in OpsDB propagate to hot-path service, results flow back as observations

9. Frontend development: consuming both APIs

The bridge pattern in detail:

The hot-path service reads configuration from a local cache that runners populate. The cache refresh cycle is configured in the runner spec — every 60 seconds, every 5 minutes, on-demand via a trigger. The hot-path service never calls the OpsDB API directly at run-time. If OpsDB is unavailable, the hot-path service continues with cached configuration. When OpsDB returns, the runner catches up.

OpsDB governed entities

- config push runner (reads entities, transforms, pushes)
- hot-path service local cache
 - hot-path service processing

hot-path service results

- observation pull runner (reads results, transforms, writes)
- OpsDB observation cache
 - search API (for dashboards, reports, reconciliation)

The observation pull runner writes results as observation cache entries with freshness timestamps. A reconciler runner can compare the observation cache against governed entities to detect discrepancies — “OpsDB says the customer’s plan is Premium but the billing engine charged them at the Basic rate.”

7.3 Operational Wrapper Pattern **Architecture position:** OpsDB wraps a specialized system with governance. The specialized system is the application. OpsDB manages configuration, accounts, policies, compliance, and audit. 10–30% governed state.

What you build: Schema YAML for the governance wrapper — accounts, configuration, policies, audit metadata. The specialized system as a purpose-built application (this is most of the engineering effort). Configuration push runners. Observation pull runners. A frontend that primarily interacts with the specialized system, with OpsDB-backed administration and compliance views.

What OpsDB provides: Governed management of configuration and policies. Audit trail for configuration changes. Compliance evidence production. Account management with full authorization model. The specialized system operates independently.

Where effort concentrates: The specialized system (60–70% of total effort). Schema design for the wrapper (10%). Runner bridges (10%). Frontend for the specialized system (10–15%). Admin/compliance frontend consuming OpsDB (5%).

Representative applications: Real-time communication (message bus is the system), stream processing (event processor is the system), ML platforms (training/inference is the system), video streaming (transcoding/delivery is the system), high-frequency trading (matching engine is the system).

Construction sequence:

1. Specialized system design and implementation (the primary engineering work)
2. Identify what configuration and operational data should be governed
3. Schema design for the governance wrapper
4. Runner bridge implementation
5. Admin frontend consuming OpsDB API
6. Compliance and audit views consuming OpsDB audit log and evidence records

7.4 Personal AppDB Pattern Architecture position: OpsDB is the primary backend on personal hardware. Single user. Auto-approve everything. Minimal runner population.

What you build: Schema YAML for personal data domains. Personal runners for external integrations (weather, fitness, banking, RSS). A simple frontend — a mobile app, a web interface, a CLI.

What OpsDB provides: Everything from the governed-state pattern, with governance features that simplify to invisibility. Change management auto-approves. Authorization is a single role. Audit logging runs silently in the background. Versioning and search are the primary value — you never lose data, and you can find anything.

Where effort concentrates: Schema design for personal domains (40%). Personal runners for integrations (30%). Frontend (30%).

Graduated formalization pattern:

Start with a generic collection entity that has a discriminated JSON payload. Track recipes, books, plants, and workout sessions as collection items with different payload schemas. When a domain becomes important enough to warrant proper schema treatment — your recipe collection grows from casual tracking to a serious reference — write a proper schema for that domain. A migration runner reads the generic collection items of that type and writes them as proper entities through change sets. The generic collection stays for casual tracking. The proper schema provides structured fields, relationships, typed queries, and specific runners.

```
# Generic collection for casual tracking
```

```
name: collection
```

```
domain: personal
```

```

fields:
  - name: name
type: varchar
length: 255
nullable: false
  - name: collection_type
type: enum
values: [recipe, book, plant, workout, wine, movie, travel]
nullable: false
  - name: item_data_json
type: json
nullable: false
  - name: tags
type: varchar
length: 1024
nullable: true

```

As the personal AppDB matures, some collections graduate to proper schemas. Others stay generic indefinitely. The decision to graduate is driven by pain — when the generic schema’s lack of typed fields, explicit relationships, or specific runners becomes a friction, it is time to formalize. This is the correct use of iterative discovery within the OpsDB architecture — the generic collection is the sandbox, and graduation to a proper schema is the point where the domain is understood well enough for comprehensive design.

7.5 Distributed Application Pattern **Architecture position:** The application is packaged for deployment by others. The development team’s AppDB is the prototype. Each deployment is a separate DOS.

What you build: The prototype AppDB — schema, policies, runners, documentation. A release packaging process that produces a deployable artifact containing the schema YAML, runner binaries, default policy configurations, and operational documentation. A

schema upgrade path from version N to version N+1 using the additive evolution rules.

What OpsDB provides: The deployment infrastructure. Each deploying organization runs their own OpsDB instance, loads the release schema, configures their own policies, and runs the packaged runners. The gate pipeline, versioning, audit, and authorization work identically in every deployment.

Release versioning:

Each release includes a schema version identifier. Schema changes between releases follow the additive evolution rules — new fields, new entities, widened constraints, no deletions, no renames, no type changes. Upgrading from version N to N+1 means applying the schema diff through the schema change set mechanism. The loader computes the diff between the deployed schema and the release schema, generates the additive changes, and applies them through a schema change set.

Release 1.0:

```
entities: project, task, comment, label

fields: (initial set)

policies: (default approval rules, retention policies)
```

Release 1.1:

```
entities: project, task, comment, label, milestone (added)

fields: task.estimated_hours (added), task.priority max widened from 5 to 10

policies: (default + milestone approval rules)
```

The upgrade from 1.0 to 1.1 adds the milestone entity, adds the estimated_hours field to task (nullable, so existing rows are unaffected), and widens the priority range. No existing data is affected. No existing consumers break. The deploying organization reviews the schema change set, approves it, and the schema executor applies it.

Per-deployment policy configuration:

The release includes default policies — default approval rules, default retention policies, default access control configuration. The deploying organization modifies these through standard change sets to match their requirements. A healthcare organization deploying a patient management AppDB sets retention policies to match medical record retention law. A small team deploying a project management AppDB sets all approval rules to auto-approve. The schema is shared. The policies are local.

8. AI-Assisted Construction Methods

The AppDB architecture is well-suited for AI assistance at every stage of construction. The closed vocabulary, mechanical verification, small artifact size, and governed change management create an environment where AI is productive and contained.

8.1 AI as Schema Generator An AI can generate schema YAML from a domain description. The prompt pattern:

1. Describe the domain in plain language — what entities exist, what their relationships are, what fields they have
2. Specify the schema conventions — singular names, lowercase underscores, FK naming, type suffixes
3. Ask for YAML output conforming to the loader's expected format

The AI generates YAML files. The loader verifies them mechanically — structural correctness, naming conventions, FK target existence, type validity, constraint completeness. If the loader rejects the output, the error message identifies the specific violation. The AI can be prompted to fix the violation, or the developer can fix it manually.

This works well because the output format is constrained and verifiable. The AI cannot hallucinate a new field type — only nine exist. It cannot hallucinate a new constraint mechanism — only six exist. It cannot generate structurally invalid YAML that passes the loader — the loader catches every structural violation. The worst case is a schema that is structurally valid but domain-incorrect — wrong field types, wrong relationships, missing entities. These errors are visible in the YAML and correctable through human review.

8.2 AI as Runner Generator An AI can generate runner code from a runner spec description. The prompt pattern:

1. Describe the runner's purpose — what it reads, what it computes, what it writes
2. Specify the get-act-set structure
3. Specify the library suite calls available
4. Ask for Go code using the library suite

The AI generates 150–300 lines of runner code. The code is small enough for complete human review. The library suite handles all infrastructure concerns, so the AI only generates domain logic — the act phase computation. Errors in the generated code are contained by the runner's scope declaration — even incorrect runner code cannot affect data outside its declared scope because the API gate and library suite enforce boundaries.

8.3 AI as Change Proposer An AI operating as a system participant — an AI runner — reads governed state, makes domain decisions, and proposes change sets through the

standard API. The AI is authenticated as a service account with declared capabilities and targets. Its proposals go through the same gate pipeline as human proposals.

Policy configuration for AI proposers:

Create approval rules that match on the AI service account's identity. Low-risk AI proposals (updating observation cache summaries, categorizing items, computing derived values) can auto-approve. Medium-risk AI proposals (updating governed entity fields based on reconciliation) require human review. High-risk AI proposals (modifying policies, access control, financial data) require multiple human approvers.

The approval rules are data rows. Adjusting how much autonomy the AI gets means changing policy rows through governed change sets. The decision of "how much should we trust the AI" is itself a governed, versioned, auditable decision.

Containment:

The AI service account's runner scope declarations limit what it can read and write. The gate pipeline validates every AI-proposed change set against the same constraints as human proposals — schema validation, bound validation, policy evaluation, authorization. If the AI proposes invalid changes, they are rejected at the gate. If the AI proposes changes outside its scope, they are rejected at authorization. If the AI proposes changes that require approval, they queue for human review.

The audit trail records every AI proposal — accepted, rejected, pending. The version history preserves the state before and after every AI-applied change. Rollback is a governed change set that restores prior values. The AI cannot make irreversible changes because the system has no irreversible changes — every state is reconstructable from version history.

8.4 AI Observation Generation AI generates derived observations over governed data, as described in Section 5.8. The key points for construction:

When to use AI observations:

Incident summarization — the AI traverses the dependency chain, recent changes, alert fires, and produces a human-readable summary of what happened and what is affected.

Data classification suggestions — the AI reads entity content and proposes access classification levels. The proposals are observations in the cache. A human reviews the suggestions and applies them through governed change sets.

Anomaly narrative — the AI reads observation cache metrics and produces natural-language descriptions of anomalies: "CPU utilization on service X increased 40% over the past hour, coinciding with a deployment change set applied at 14:30."

Compliance gap analysis — the AI reads the compliance regime entities, the evidence records, and the governed entity configurations, and produces a narrative identifying gaps: "No evidence record exists for backup verification on database Y for the past 30 days, which is required by SOC2-Availability."

When not to use AI observations:

For factual queries that the search API answers directly. “How many tasks are overdue?” is a search query, not an AI observation. For computations that must be exact — financial totals, inventory counts, compliance metrics. These are custom domain runners with deterministic logic, not LLM calls with probabilistic output.

9. Construction Method Catalog

Each method in this paper is cataloged below for reference. The catalog provides the method’s applicability, inputs, and outputs.

```
methods(id|name|section|applies_to|inputs|outputs)
```

```
# Domain Analysis Methods
```

```
M01|Entity Enumeration|2.1|all positions|domain knowledge|entity list with i
```

```
M02|Field Enumeration|2.2|all positions|entity list|field list with types, co
```

```
M03|Relationship Classification|2.3|all positions|entity list|classified rel
```

```
M04|Lifecycle Identification|2.4|entities with state|entity list|state enums
```

```
M05|Policy Enumeration|2.5|all positions|entity list + relationships|approva
```

```
M06|Schedule Enumeration|2.6|all positions|domain knowledge|recurring operat
```

```
M07|External Integration Enumeration|2.7|split + wrapper|domain knowledge|au
```

```
M08|Hot Path Identification|2.8|split + wrapper|domain knowledge + performanc
```

```
M09|Architecture Position Selection|2.8|all|domain analysis results|position
```

```
# Schema Construction Methods
```

```
M10|Entity Design|3.1|all positions|entity from M01 + fields from M02|entity
```

```
M11|Type Selection|3.2|all positions|field from M02|type + constraints from o
```

```
M12|Constraint Selection|3.3|all positions|field from M02|bounds, lengths, e
```

```
M13|Discriminator Pattern|3.4|entities with typed variants|entity with subty
```

```
M14|Bridge Table Design|3.5|many-to-many + polymorphic relationships|relatio
```

M15|Hierarchy Design|3.6|self-referential entities|entity with parent-child relationship|self-FK + depth field + traversal runner

M16|Governance Field Configuration|3.7|entities needing access control|access

M17|Draft Mode Configuration|3.8|interactive editing entities|property analy

M18|Schema Organization|3.9|all positions|complete entity set|directory stru

Policy Construction Methods

M19|Approval Rule Design|4.1|all governed entities|entity types + sensitivity

M20|Access Control Design|4.2|all positions|data sensitivity map|roles, group

M21|State Transition Rules|4.3|entities with lifecycles|transition graph from

M22|Retention Policy Design|4.4|all governed entities|regulatory + operation

M23|Cross-Field Invariant Rules|4.5|entities with conditional constraints|do

Runner Construction Methods

M24|Runner Design|5.1|all runners|domain logic requirement|runner spec with i

M25|Puller Runner|5.2|external data import|authority + external API|observat

M26|Reconciler Runner|5.3|desired vs observed correction|governed entities +

M27|Verifier Runner|5.4|scheduled verification|schedules + targets|evidence

M28|Notification Runner|5.5|state transition alerting|change sets + schedule

M29|Configuration Push Runner|5.6|hot path config delivery|governed entities

M30|Observation Pull Runner|5.7|hot path result collection|external system re

M31|AI Observation Runner|5.8|LLM-derived observations|governed entities + c

M32|Custom Domain Runner|5.9|domain-specific computation|governed entities +

M33|Server Runner Configuration|5.10|runner thread management|runner specs|t

Frontend Integration Methods

M34|Authentication Integration|6.1|all frontends|IdP configuration|SSO token

M35|Read Patterns|6.2|all frontends|search API|list views, detail views, acti
in-time views

M36|Write Patterns|6.3|all frontends|change set API|standard writes, auto-
approved, approval-required, creation, bulk, emergency

M37|Draft Mode Integration|6.4|draft-mode entities|draft mode API|autosave +

M38|Multi-Tenancy Integration|6.5|multi-user applications|authorization API
filtered results, field omissions, group-scoped navigation

Application Pattern Methods

M39|Governed-State-Dominant Construction|7.1|primary backend position|full c

M40|Split-Backend Construction|7.2|split backend position|domain analysis +

M41|Operational Wrapper Construction|7.3|wrapper position|specialized system

M42|Personal AppDB Construction|7.4|personal scale|personal data domains|per

M43|Distributed Application Construction|7.5|packaged applications|prototyp
deployment policy

AI-Assisted Methods

M44|AI Schema Generation|8.1|all positions|domain description + conventions|
verifiable YAML files

M45|AI Runner Generation|8.2|all runners|runner spec description + library su
300 line runner code

M46|AI Change Proposer Configuration|8.3|governed entities|AI runner spec + a

M47|AI Observation Configuration|8.4|governed entities with summary needs|ru

10. Decision Trees

For each major construction decision, a decision tree identifies the method to apply.

10.1 Architecture Position Selection

Does the application have processing that cannot tolerate the gate pipeline la

- | — No → Is it a personal or single-user application?
- | | — Yes → Personal AppDB (M42)
- | └ No → Is it intended for distribution as a deployable package?
- | | — Yes → Distributed Application (M43) with Governed-State-Dominant (M39)
- | └ No → Governed-State-Dominant (M39)
- └ Yes → What percentage of the data model is governed state?
- └ 70%+ → Split-Backend (M40)
- └ 10-70% → Operational Wrapper (M41)
- └ <10% → OpsDB may not be the right tool; evaluate metadata-only position

10.2 Entity Design Selection

Does the entity have subtypes with different field sets?

- | — Yes → Discriminator Pattern (M13)
- └ No → Does the entity relate to multiple different entity types polymorphically?
- | — Yes → create bridge tables per target type (M14)
- └ No → Does the entity have a parent-child hierarchy?
- | — Yes → Self-referential FK + depth field (M15)
- └ No → Standard Entity Design (M10)

10.3 Runner Kind Selection

What does the runner do?

- | — Imports data from external system → Puller (M25)
- | — Compares desired vs observed and corrects → Reconciler (M26)

- |— Verifies conditions and produces evidence → Verifier (M27)
- |— Dispatches notifications on state transitions → Notification (M28)
- |— Pushes governed config to external system → Config Push (M29)
- |— Pulls results from external system → Observation Pull (M30)
- |— Generates LLM-derived summaries → AI Observation (M31)
- └— Domain-specific computation → Custom Domain (M32)

10.4 Gating Mode Selection

What does the runner write?

- |— Observation cache (external state, metrics, AI summaries)
 - | → Direct write (authenticated, validated, audited, no change set)
 - |— Governed entity changes with low risk
 - | → Auto-approved change set (change set created, auto-transitions)
 - |— Governed entity changes with high risk or compliance requirements
 - | → Approval-required change set (routes to human approvers)
 - └— Mixed (different targets with different stakes)
- Per-target gating (direct write for observations, approval for governed)

10.5 Governance Flag Decision

Is this entity used for interactive editing where per-keystroke versioning creates noise?

- |— No → Full governance (default)
- └— Yes → Does the entity hold compliance-relevant data?

|— Yes → Full governance (compliance requires per-write auditability)

└— No → Does the entity hold financial data?

|— Yes → Full governance (financial data requires per-write versioning)

└— No → Property analysis: which properties are weakened by draft mode?

|— Per-write versioning → acceptable if committed versions suffice

|— Per-write audit → acceptable if committed version audit suffices

└— Change management → acceptable if auto-commit governance suffices

→ Enable draft mode flags + document tradeoff in schema notes (M17)

10.6 AI Assistance Selection

What task are you performing?

|— Schema design from domain description → AI Schema Generation (M44)

|— Runner implementation from spec → AI Runner Generation (M45)

|— Automated domain decisions that modify governed state → AI Change Proposer

|— Derived summaries over governed data → AI Observation (M47)

└— Domain analysis, policy design, architecture decisions → Human judgment
(AI can assist with enumeration but the decisions are engineering)

11. Cross-Reference to Taxonomy

Each construction method connects to the underlying taxonomy's mechanisms, properties, and principles.

method_to_taxonomy(method|mechanisms|properties|principles)

M01|AF1 (Storage)+AF13 (Representation) |P04 (Consistency) |R05 (Comprehensive) +

M02 | AF13 (Representation) | P04 (Consistency) + P08 (Determinism) | R01 (Data Primacy)
M03 | AF1 (Storage) + AF4 (Selection) | P04 (Consistency) + P05 (Integrity) | R05 (Compre
M04 | NM01 (State Machine Evaluator) | NP01 (Lifecycle Integrity) | NR02 (Rules as Da
M05 | AF2 (Gating) + AF9 (Coordination) | P06 (Authenticity) + P07 (Confidentiality) + P
M06 | AF8 (Lifecycle) + NM04 (Temporal Projection) | NP02 (Temporal Consistency) | R01
M07 | AF5 (Information Movement) | P11 (Availability) + P19 (Failure Transparency) | R
M08 | AF5 + AF10 + AF11 + AF12 | P10 (Liveness) + P11 (Availability) + P12 (Boundedness) | R1
M10 | AF1 + AF13 | P04 + P08 | R01 + R03 (Convention over Lookup)
M13 | AF13 + NM02 (Rule Engine) | P04 + P08 | R01 + R04 (0/1/Infinity)
M14 | AF1 + AF4 | P04 + P05 | R05 + R06 (One Way)
M17 | varies per flag | weakens P20 (Observability) + P21 selectively | NR03 (Separate
M19 | AF2 + AF9 | P06 + P21 + NP05 (Non-repudiation) | R18 + NR02
M20 | AF2 | P06 + P07 + P13 (Isolation) | R09 (Fail Closed) + R18
M21 | NM01 | NP01 | NR02
M24 | AF6 (Control Loop) + AF7 (Sensing) | P01 (Idempotency) + P10 + P12 | R07 (Idempotent
Triggered) + R11 (Bound Everything)
M25 | AF5 + AF7 | P03 (Durability) + P11 | R08 + NR01 (Separate Domain from Infra)
M26 | AF6 + AF7 | P01 + P04 + P09 (Convergence) | R08 + R07
M27 | AF7 + AF8 | P20 + P21 | R21 (Make State Observable)
M28 | AF5 | P10 + P11 | R18 + R19 (Push Decision Down)
M29 | AF5 + AF10 | P11 + P19 | R17 + R20 (Push Work Down)
M30 | AF5 + AF7 | P03 + P11 | R17 + R08
M31 | AF5 + AF10 + NM03 (Accumulator) | P11 + SP04 (Domain Opacity) | NR01 + SM04 (Semantic

M32|AF10+NM02+NM03|NP03 (Computational Correctness) |NR01+NR02

M33|AF12 (Allocation) +AF9|P10+P12+P13|R04+R11+SM06 (Scene as Isolation)

M39|all governed mechanisms|all governed properties|all universal principles

M40|governed + AF5+AF11|governed + P11+P19|R14+R17+R19

M41|AF1+AF2+AF5+AF7|P06+P07+P20+P21|R14+R17+NR01

M44|AF13+SM04|SP01 (Vocabulary Closure)+SP02 (Structural Security) |NP02 (Vocab

M46|AF2+AF6+SM07 (Structured Trace) |P01+P21+SP03 (Hot-Swap Safety) |NP01 (Security by Anatomy)+NP04 (Infra Fix Protects All)

12. Boundaries and Limitations

The construction methods in this paper apply within the OpsDB abstraction boundary. Several limitations constrain what can be built.

The closed vocabulary is a real constraint. Nine types, six constraints, three modifiers. If your domain requires a constraint that the vocabulary cannot express — a field that must match a specific pattern, a constraint that depends on external state, a type not in the nine — you cannot express it in the schema. Cross-field invariants go to policy rows. Complex validation goes to runner logic that runs before proposing change sets. But some validation requirements may not fit cleanly.

Schema evolution is additive only. You cannot delete fields, rename entities, or change types. The six-step duplication pattern handles type changes but at the cost of permanent tombstone fields. Over years, schemas accumulate deprecated fields. This is a real cost — the schema grows monotonically. The tradeoff is that consumers, version history, audit logs, and runner logic never break from schema changes.

The gate pipeline adds latency. Every write passes through ten steps. For human-pace operations this is invisible. For high-frequency automated writes — thousands per second — the pipeline is the bottleneck. Observation cache tables with appropriate governance flags reduce this for observation-like data. For truly high-frequency requirements, a hot-path service outside OpsDB is necessary.

OpsDB is not a compute platform. Runners perform computation, but heavy computation — ML training, video transcoding, scientific simulation — belongs in specialized systems. OpsDB governs the configuration and collects the results. The computation itself runs elsewhere.

Discovery-phase prototyping is costly. The additive-only evolution rules penalize schema experimentation. If you do not yet know your data model, the cost of guessing wrong is permanent tombstone fields. The graduated formalization pattern (generic

collection → proper schema) mitigates this for some domains. For others, prototyping in a less rigid system and migrating to OpsDB once the domain stabilizes is the right approach.

The frontend is your responsibility. OpsDB provides the data substrate. It does not render pages, serve static assets, manage client-side state, handle responsive layout, or implement user interaction patterns. The frontend is a conventional development effort of conventional size. OpsDB reduces backend work by an order of magnitude. It does not reduce frontend work.

13. Summary

This paper enumerates 47 construction methods for building applications with OpsDB Application Architecture. The methods cover domain analysis (9 methods), schema construction (9 methods), policy construction (5 methods), runner construction (10 methods), frontend integration (5 methods), application patterns by type (5 methods), and AI-assisted construction (4 methods).

The methods compose. A governed-state-dominant application uses methods M01–M06 for domain analysis, M10–M18 for schema construction, M19–M23 for policy construction, M24–M28 plus M31–M32 for runners, M34–M38 for frontend integration, and M44–M47 for AI assistance. A split-backend application adds M08, M29, M30, and M40. A personal AppDB uses M42 with graduated formalization. A distributed application adds M43 with release versioning.

The construction sequence for every application type follows the same fundamental pattern: analyze the domain comprehensively, declare it as schema, verify it through the loader and API, implement domain-specific logic as runners, and connect a frontend. The infrastructure — validation, authorization, versioning, change management, audit, search, retention — exists from the first entity definition. The developer’s effort concentrates on understanding the domain and declaring it correctly. The infrastructure handles everything else.

The methods are grounded in the three-axis taxonomy from the infrastructure specification: mechanisms determine what the application does with data, properties determine what guarantees it needs, and principles determine how it is assembled. Each method in the catalog traces to specific mechanisms, properties, and principles. The construction decisions are engineering decisions made on explicit axes, not fashion-driven choices copied from blog posts.

The architecture rests on data primacy: the schema is data, the policies are data, the business rules are data, the runner configurations are data, the AI observations are data. Code is fixed infrastructure that interprets data, or small runners that make domain decisions within declared bounds. The developer’s long-lived artifact is the schema. Everything else — runners, frontends, infrastructure versions — is replaceable. The data survives.

Appendix A: Entity Design Pattern Reference

entity_patterns	id	pattern	description	when_to_use	yaml_elements	example
EP01	Standard Entity	single entity type with typed fields and constraints	most common			
EP02	Discriminator Entity	entity with subtype-specific typed JSON payload	entity has variants with different field sets sharing type JSON schemas	monitor (prometheus_query, http_probe, tcp_probe), scheduler		
EP03	Bridge Entity	join table connecting two entity types	many-to-many relationships; polymorphic attachment	two mandatory FK fields + optional		
EP04	Polymorphic Bridge Set	multiple bridge tables connecting one source to multiple target pair	each with two mandatory FKs	task_comment + project_comment + milestone		
EP05	Hierarchical Entity	self-referential entity with parent chain	tree structure	FK (parent_*_id) + depth int field + hierarchy runner	category, organizational	
EP06	Governance Scoped Entity	entity with per-row access control	entities visible only to specific groups or with field-level sensitivity	_requires_group FK + _access_classification FK	patient_rec	
EP07	Draft Mode Entity	entity with relaxed interim governance	interactive editing	keystroke versioning creates noise	draft_mode flags (_autoversion_disabled, ...)	
EP08	Observation Cache Entity	non-governed entity holding external or derived				
EP09	Evidence Record Entity	append-oriented entity recording verification	records			
EP10	Schedule Entity	entity representing recurring or deadline-driven work	anything that recurs, expires, or has temporal triggers	discrimin		
EP11	Policy Entity	entity expressing behavioral rules evaluated by gate pipeline				
EP12	Authority Entity	entity representing an external system connection	external			
EP13	Runner Spec Entity	entity declaring a runner's configuration and bounds				
EP14	Generic Collection Entity	catch-all entity for casual tracking with gradual stage domain exploration	collection_type enum + item_data_json + tags	collect		

Appendix B: Field Type Decision Matrix

type_selection	condition	type	constraints_to_declare	notes
whole number with domain bounds	int	min, max	always declare both bounds	use 1 for min
decimal number with domain bounds	float	min, max	always declare both bounds	use 1 for min
short bounded string (names, titles, codes)	varchar	length	63 for identifiers	
long unbounded text (prose, markdown, notes)	text	none	use sparingly	most strings are short
true/false present state	boolean	none	prefix is_ (is_active, is_published)	
true/false past event	boolean	none	prefix was_ (was_approved, was_escalated)	
point in time	datetime	none	suffix _time; stored UTC; timezone handling in runtime	
calendar day without time	date	none	suffix _date; use when time component is not needed	
constrained value from fixed set	enum	values[]	values can be added never removed	
subtype-specific structured data	json	none	(validated by discriminator-selected JSON schema) always pair with discriminator enum field	
reference to another entity	foreign_key	references (target table)	name as ref_	

Appendix C: Constraint Selection Matrix

constraint_selection	constraint_type	applies_to	declaration	enforcement_position
numeric range	int, float	min, max	gate step 4 (bound validation)	can widen (lower min, higher max)
string length	varchar	length	gate step 4	can widen; never narrow
enum value set	enum	values[]	gate step 4	can add values; never remove
foreign key reference	foreign_key	references	gate step 4 (existence check)	target table
nullable	all types	nullable: true/false	gate step 3 (schema validation)	can change from not-null to nullable (widening); never from nullable to not-null

unique|all types except text and json|unique: true|gate step 4|can add to empty

Appendix D: Relationship Pattern Matrix

relationship_patterns(pattern|cardinality|implementation|fk_nullable|naming convention|ownership|one-to-many mandatory|FK on child entity|false|parent_table_id|task optional reference|one-to-many optional|FK on referencing entity|true|reference multiple references to same target|one-to-many multiple|multiple FKs with roles many-to-many|many-to-many|bridge table with two mandatory FKs|false (both)|task polymorphic attachment|one-to-many across types|one bridge table per target type self-referential hierarchy|tree|nullable self-FK + depth field|true (parent)

Appendix E: Governance Field Reference

governance_fields(field|type|purpose|gate_layer|default|when_to_use)

_requires_group|FK → ops_group|restrict entity visibility to group members|layer 3 (entity governance)|null (no restriction)|project-scoped data, team-scoped data, personal data

_access_classification|FK → data_classification|control field-level visibility by sensitivity|layer 3 (per-field classification)|null (no restriction)|salary data, medical details, financial data

versioned: true|schema flag|enable change management and version history|step 4

_autoversion_disabled|draft mode flag|skip version row on interim saves|step 4

_edit_latest_version|draft mode flag|write directly to current row, skip change row|step 4

_audit_logs_disabled|draft mode flag|skip audit log on interim saves|step 8|false

Appendix F: Policy Type Reference

```
policy_types(id|type|purpose|evaluated_at|match_criteria|effect)

PT01|approval_rule|determine who must approve a change set|gate step 7|entity
PT02|state_transition|declare valid state machine transitions|gate step 5|en
PT03|semantic_invariant|enforce cross-field conditional constraints|gate ste
PT04|retention|declare per-entity-type retention horizons|reaper runner read
PT05|access_control|declare role-to-entity-type permissions|gate step 2 laye
PT06|change_management|declare emergency review windows and SoD rules|gate st
PT07|schedule_governance|declare scheduling constraints for runners and oper
PT08|security_zone|declare environment-level access restrictions|gate step 2
PT09|data_classification|declare sensitivity levels and access grants|gate s
```

Appendix G: Runner Kind Reference

```
runner_kinds(id|kind|purpose|reads_from|writes_to|gating_mode|trigger|typi

RK01|Puller|import external data|external API + runner spec + authority|obser
400

RK02|Reconciler|compare desired vs observed, correct drift|governed entities
approve or approval-required|scheduled|200-350

RK03|Verifier|check conditions, produce evidence|schedule + target entities +
250

RK04|Notification|detect transitions, dispatch messages|change_sets + schedu
driven|200-350

RK05|Change-set Executor|apply approved change sets|approved change_sets|ent
approval direct|event-driven (new approvals)|150-250

RK06|Reaper|enforce retention policies|retention_policy rows|deletes/soft-
deletes on expired rows|direct write|scheduled|150-200
```

RK07|Config Push|push governed state to external system|governed entities + r
change|200-350

RK08|Observation Pull|pull results from external system|external API + runner
300

RK09|AI Observation|generate LLM-derived summaries|governed entities + conte
change|200-350

RK10|Reactor|process external events|webhook/event + runner spec|observation
driven|200-300

RK11|Drift Detector|find discrepancies without correcting|desired entities +
required|scheduled|200-300

RK12|Scheduler|manage execution timing for other runners|schedule entities +
approve|scheduled|150-250

RK13|Bootstrapper|provision new entities from templates|template config + tri
approve or approval|event-driven|200-350

RK14|Custom Domain|domain-specific computation|governed entities + domain ru
300

Appendix H: Gating Mode Decision Matrix

gating_modes (mode|change_set_created|approval_routing|audit_logged|version.
direct_write|no|no|yes|no|observation cache writes, evidence records, runner
auto_approve|yes|auto-transitions to approved|yes|yes (if versioned)|low-
risk governed entity changes, routine automated updates, personal AppDB all ch
approval_required|yes|routes to human approvers|yes|yes (on apply)|high-
risk changes, financial data, access control, compliance-
relevant, AI-proposed governed changes
post_approval_direct|no (already approved)|already completed|yes|yes|change
set executor applying previously approved change sets
emergency|yes (reduced)|reduced approval count + mandatory flag|yes (with eme
glass situations with mandatory post-incident review

Appendix I: Schedule Type Reference

`schedule_types(id|type|payload_fields|use_when|example)`

ST01|`cron_expression`|`cron_string`, `timezone`|fixed recurring schedule with specified frequency
daily at 2 AM

ST02|`rate_based`|`interval_seconds`|simple periodic execution|every 300 seconds

ST03|`event_triggered`|`trigger_entity_type`, `trigger_field`, `trigger_condition`|

ST04|`calendar_anchored`|`anchor_date`, `recurrence_rule`, `timezone`|business calendar

ST05|`deadline_driven`|`deadline_field`, `entity_type`, `warning_threshold_days`|applies to deadlines

ST06|`manual`|`authorized_roles`, `required_evidence_type`|human-initiated scheduled operations|physical inspection; vendor review; tape rotation

Appendix J: Draft Mode Property Impact Matrix

`draft_mode_impact(flag|property_affected|impact|still_enforced|risk_level)`

`_autoversion_disabled`|per-write versioning (P16 ordering)|interim states not available
recovery granularity reduced to committed versions

`_autoversion_disabled`|point-in-time reconstruction|reconstruction available
acceptable for editing use cases

`_edit_latest_version`|change management (approval routing)|interim saves bypassed
behavioral changes between commits are ungoverned

`_edit_latest_version`|attribution of interim changes|individual interim saves not available
interim saves attributed via auth identity in request

`_audit_logs_disabled`|auditability (P21)|interim saves not individually recorded
interim activity invisible to audit queries

`_audit_logs_disabled`|compliance evidence|gap in audit trail between committed and interim
version compliance|high for compliance-relevant data – do not use on regulated entities

all three combined|combined weakening|interim editing is structurally validated, managed, or audited|auth + authz + schema validation + bound validation + policy regulated interactive editing entities

Appendix K: Schema Evolution Rule Reference

evolution_rules(id|change_type|allowed|conditions|mechanism|risk)

SE01|add nullable field|yes|field must be nullable; no default required for existing rows unaffected

SE02|add entity type|yes|no conditions beyond naming conventions|loader creates no existing consumers affected

SE03|add enum value|yes|new value appended to set|loader updates check constraints existing rows retain current values

SE04|widen numeric range|yes|new min $\leq$ old min AND new max $\geq$ old max|loader updates existing valid values remain valid

SE05|widen string length|yes|new length $\geq$ old length|loader updates column length existing valid values remain valid

SE06|add index|yes|no conditions|loader creates index|temporary – index build may lock table briefly

SE07|delete field|forbidden|never|n/a|breaks version history, audit log, consumers

SE08|rename field or entity|forbidden|never|n/a|breaks every consumer by name

SE09|change field type|forbidden|never|use six-step duplication pattern|breaks consumers expecting prior type

SE10|narrow numeric range|forbidden|never|existing rows may violate|breaks existing

SE11|remove enum value|forbidden|never|existing rows may hold removed value|breaks

SE12|tighten uniqueness|forbidden|never|existing rows may violate|breaks existing

SE13|add not-null field|forbidden (unless empty table)|only on empty table because

duplication_pattern_steps(step|action|duration|notes)

DP01	add new field alongside old immediate new field is nullable; old field un
DP02	begin writing to both fields deployment all writers update both old and n
DP03	migrate readers to new field gradual readers switch from old field to new
DP04	mark old field deprecated change set notes field documents deprecation
DP05	continue writing both for safety period configurable ensures all readers
DP06	stop writing to old field deployment old field becomes tombstone; never

Appendix L: Library Suite Call Reference

	library_calls(id library call purpose runner_phases handles)
LC01	opsdb-api search query governed entities with filters, joins, projection
LC02	opsdb-api get_entity fetch single entity with optional joins get auth, level access filtering
LC03	opsdb-api get_entity_at_time reconstruct entity state at timestamp get
LC04	opsdb-api get_entity_history fetch version chain for entity get auth, p
LC05	opsdb-api submit_change_set propose field changes with reason set auth,
LC06	opsdb-api bulk_submit propose multi-entity atomic changes set auth, val
LC07	opsdb-api write_observation write to observation cache tables set auth,
LC08	opsdb-api emergency_apply break-glass change with reduced approval set
LC09	k8s list_resources query Kubernetes API for resources get auth, paginat
LC10	k8s apply_manifest apply Kubernetes manifest act auth, dry-run support, retry, conflict handling
LC11	k8s watch_events subscribe to Kubernetes event stream get auth, reconne
LC12	cloud-aws describe_resources query AWS API for resource state get auth,
LC13	cloud-aws apply_change modify AWS resource act auth, retry, idempotency

run support

LC14|cloud-gcp|describe_resources|query GCP API for resource state|get|auth,

LC15|cloud-gcp|apply_change|modify GCP resource|act|auth, retry, operation p

LC16|secrets|read_secret|fetch secret value from vault|act|auth, lease manag

LC17|secrets|rotate_secret|generate and store new secret|act|auth, atomic sw
version retention

LC18|notification|send_email|dispatch email via configured SMTP|act|auth, re

LC19|notification|send_webhook|dispatch webhook to configured URL|act|auth,

LC20|notification|send_chat|dispatch message to chat platform|act|auth, retr

LC21|logging|structured_log|emit structured log entry with correlation ID|al

LC22|logging|metric|emit metric data point|all phases|metric naming, label pr

LC23|git|clone_repo|clone git repository|get|auth, shallow clone, sparse che

LC24|git|commit_and_push|commit changes and push|act|auth, retry, conflict d

LC25|template|render|render template with variable substitution|act|no logic

LC26|llm|generate|call LLM API with prompt and parameters|act|auth, retry, ti

LC27|lifecycle|heartbeat|report runner thread health|set|cycle timing, error

LC28|lifecycle|claim_work|claim work item via advisory lock|get|lock acquisi

LC29|lifecycle|release_work|release work item claim|set|lock release, comple

Appendix M: AI Runner Configuration Reference

ai_runner_config(field|type|purpose|example_values)

model|varchar|LLM model identifier|"claude-sonnet", "claude-
haiku"

max_tokens|int|maximum response tokens from LLM|100, 500, 2000

character_budget|int|target character count for output|12, 80, 500, 2000, 5000
 regeneration_trigger|enum|when to regenerate observation|source_entity_change
 trigger_schedule|varchar|cron or rate for scheduled regeneration|"*/15 * * * *"
 target_entity_types|json array|which entity types to summarize|["incident"],
 context_traversal|json array|relationship paths to traverse for context assembly
 > affected_service", "incident -> change_set"]
 prompt_template_ref|varchar|reference to prompt template entity|"incident_summary"
 observation_cache_table|varchar|target observation cache table|"observation_cache"
 observation_key_prefix|varchar|prefix for observation cache keys|"ai_summary_"
 retry_budget|int|max LLM call retries per cycle|3
 execution_timeout_seconds|int|max time for LLM call including retries|60, 120
 scope_per_cycle|int|max entities to process per runner cycle|10, 50, 100

 ai_observation_granularities(id|name|character_budget|use_case|prompt_style|granularity)
 AG01|ticker_headline|80|dashboard ticker, mobile notification header|terse sentence|all users
 AG02|severity_tag|12|severity classification label|single-word or short-phrase classification|automated routing
 AG03|brief_summary|500|notification body, list view annotation|concise paragraph|all users
 AG04|standard_summary|2000|war room display, incident detail view|structured paragraph with context|incident responders
 AG05|detailed_analysis|5000|post-incident review, compliance narrative|thorough analysis|all users
 AG06|structured_classification|100|data classification suggestion, category formatted classification with confidence|AI change proposer pipeline

Appendix N: Architecture Position Quick Reference

position_selection(position governed_ratio hot_path opsdb_role runner_brid					
AP01	Primary backend	95-100%	none	full backend	n/a OpsDB API only CRM, HR, co
AP02	Split backend	70-90%	bounded specific path	governance + catalog	config p path API e-commerce, booking, billing, CMS, IoT fleet, API gateway config, wor based games
AP03	Operational wrapper	10-30%	dominant	config + accounts + policies + audit path; OpsDB for admin	real-time chat, streaming, ML platform, video streaming
AP04	Metadata manager	5-10%	entire system	structured pointers + operational m series DB, search engine, graph DB, object store, message broker	
AP05	Personal	99%	none	full backend on personal hardware	optional personal in
AP06	Distributed	matches inner position	matches inner position	prototype + pe deployment instances	same as inner position same as inner position packaged S

Appendix O: Naming Convention Quick Reference

naming_conventions(element convention rule examples anti_examples)				
entity name	singular lowercase underscore	singular noun or noun phrase	task,	
field name	lowercase underscore	descriptive noun or adjective	title, start_d	
foreign key	referenced_table_id	target table name + _id suffix	project_id, as	
role-disambiguated FK	role_table_id	role prefix + target table + _id	vendor_c	
datetime field	*_time suffix	field name + _time	created_time, approved_time,	
date field	*_date suffix	field name + _date	due_date, birth_date, effective_c	
present-state boolean	is_* prefix	is_ + adjective or state	is_active, is_publ	
past-event boolean	was_* prefix	was_ + past participle	was_approved, was_esc	
governance field	_ prefix	underscore + field name	_requires_group, _access_c	

bridge table|source_target|both entity names singular|task_label, project_me

discriminator enum|*_type|entity_name + _type|cloud_resource_type, monitor_t

discriminator payload|*_data_json|entity_name + _data_json|cloud_data_json,

versioning sibling|*_version|entity_name + _version (auto-generated)|task_version, policy_version|task_history, task_versions

hierarchical prefix|specific to general|longer names build on shorter|web_sit

domain directory|NN_name|zero-padded number + domain name|01_identity, 02_co

Appendix P: Forbidden Pattern Quick Reference

forbidden_patterns(id|pattern|where|why_forbidden|alternative|detection)

FP01|regex in schema|schema YAML|catastrophic backtracking; dialect variation

FP02|embedded logic in schema|schema YAML|makes validation non-deterministic|all values are literals; computed values via runners|loader rejects

FP03|conditional constraints in schema|schema YAML|cross-field logic belongs in policy evaluation|semantic_invariant policy rows at ga

FP04|inheritance in schema|schema YAML|creates coupling between entity types in|loader rejects

FP05|templating in schema|schema YAML|variation via template variables create deployment divergence|one schema per AppDB; variation via runtime config data

FP06|imports within entity files|schema YAML|entity files must be leaf-level|only directory.yaml imports; entity files are self-contained|loader rejects

FP07|field deletion|schema evolution|breaks version history, audit log, all o

FP08|field/entity rename|schema evolution|breaks every consumer by name|add r

FP09|type change|schema evolution|breaks consumers expecting prior type|six-step duplication pattern|loader rejects (diff check)

FP10|range narrowing|schema evolution|existing rows may violate new bound|wic

FP11|enum value removal|schema evolution|existing rows may hold removed value

FP12|uniqueness tightening|schema evolution|existing rows may violate constr

FP13|runner invoking runner|runner design|creates orchestrator coupling and c

FP14|persistent state outside OpsDB|runner design|other runners and queries c
memory for one cycle only|code review discipline

FP15|reinventing shared libraries|runner design|divergent failure modes acro

FP16|logic in templates|templating|templates become opaque code|logic in upst

FP17|secrets in OpsDB|data boundaries|OpsDB not designed for need-
to-know + audit-on-read|authority pointers to vault; library accesses at runt

FP18|OpsDB as runtime dependency|architecture|services fail if OpsDB unreacha

FP19|side tables|schema discipline|first step toward fragmentation|absorb in

FP20|side channels|architecture|bypasses API; governance becomes advisory|AP

FP21|shared accounts|identity|defeats attribution|one identity per human; sc

FP22|draft mode on compliance data|governance flags|creates audit gap on regu
relevant tables|schema steward review

FP23|AI writing governed state directly|AI integration|bypasses human oversig

Appendix Q: Compliance Mapping by Method

compliance_by_method(method_id|method_name|compliance_properties_provided|

M10|Entity Design|validated writes, typed constraints, FK integrity|SOC2-
Processing-Integrity (CP03), ISO27001-A.8 (CP08)

M16|Governance Fields|per-entity access control, field-
level classification|SOC2-Confidentiality (CP04), PCI-
DSS-7 (CP12), HIPAA-Security (CP15)

M19|Approval Rules|attributed change approval, segregation of duties|SOC2-
Security (CP01), SOX-IT-General-Controls (CP18)

M20|Access Control|role-based + entity-scoped + field-classified access|SOC2-Security (CP01), ISO27001-A.9 (CP09), PCI-DSS-7 (CP12), HIPAA-Security (CP15)

M22|Retention Policies|configurable retention per entity type per regulation|Privacy (CP05), GDPR-Article-30 (CP17)

M27|Verifier Runner|continuous evidence production with pass/fail records|PCDSS-11 (CP14), SOC2-Availability (CP02), ISO27001-A.12 (CP10)

M35|Read Patterns (audit views)|point-in-time reconstruction, change attribution|DSS-10 (CP13), ISO27001-A.16 (CP11)

M36|Write Patterns (change sets)|full change trail with proposer, approver, reviewer|Processing-Integrity (CP03), SOX-IT-General-Controls (CP18)

M46|AI Change Proposer|AI proposals subject to same governance as human proposals|AI changes governed identically to human changes

Appendix R: Construction Effort Estimates by Application Type

```

effort_estimates(app_type|position|schema_yaml_lines|runner_count|runner_time)
project_management|AP01|2000|4|800|30|15|120|8:1
crm|AP01|3000|5|1200|50|20|180|9:1
hr_platform|AP01|3000|4|900|40|18|150|8:1
compliance_platform|AP01|2500|4|900|60|18|135|7:1
inventory_management|AP01|2000|3|600|25|12|105|9:1
personal_finance|AP01|1500|3|700|10|8|60|8:1
recipe_app|AP01|1000|1|200|5|4|45|11:1
education_platform|AP01|2500|4|900|35|16|135|8:1
legal_case_mgmt|AP01|2500|4|900|40|16|120|8:1

```

procurement	AP01	2000	3	600	35	12	105	9:1
e_commerce	AP02	3000	6	1500	50	30+hot_path	240	varies by hot path
booking_system	AP02	2000	4	1000	30	20+hot_path	120	varies by hot path
cms	AP02	2500	4	1000	35	22+hot_path	150	varies by hot path
subscription_billing	AP02	3000	6	1500	50	28+hot_path	165	varies by hot path
iot_fleet_mgmt	AP02	2500	5	1200	40	25+hot_path	150	varies by hot path
chat_platform	AP03	2000	4	1000	30	15+specialized	210	mostly specialized sys
ml_platform	AP03	2500	5	1200	35	18+specialized	135	mostly specialized syste
game_backend	AP03	2500	5	1200	30	18+specialized	180	mostly specialized syst
video_streaming	AP03	2500	5	1200	35	18+specialized	195	mostly specialized s
ad_auction	AP03	2500	5	1200	40	18+specialized	165	mostly specialized system

Appendix S: Method Composition by Application Type

method_composition	(app_type domain_analysis schema_construction policy_con
governed_state_dominant	M01-M06, M09 M10-M18 M19-M23 M24, M25-M28, M31-M33 M34-M38 M44-M47 M39
split_backend	M01-M09 M10-M18 M19-M23 M24-M33 M34-M38 M44-M47 M40
operational_wrapper	M01-M03, M05, M07-M09 M10-M12, M16, M18 M19-M20, M22 M24, M29-M31, M33 M34-M36, M38 M44, M46-M47 M41
personal	M01-M04, M06 M10-M14, M17-M18 M19 (auto-approve all), M22 M24-M25, M28, M31-M32 M34-M37 M44-M45, M47 M42
distributed	same as inner position same as inner + release versioning same as

Appendix T: Search API Operation Reference

`search_operations(operation|purpose|parameters|returns|bounded_by|use_case)`
`search|query entities with filters|entity_type, filters (AND/OR/NOT), fields`
`get_entity|fetch single entity|entity_type, entity_id, join_paths (optional)`
`get_entity_at_time|reconstruct state at timestamp|entity_type, entity_id, timestamp`
`get_entity_history|fetch version chain|entity_type, entity_id, page_size, cursor`
`get_dependencies|traverse relationships|entity_type, entity_id, join_path, context`
`search (view: with_history)|current state plus version chain|same as search + view`
`search (view: at_time)|reconstructed state at timestamp|same as search + view`

`filter_predicates(operator|description|field_types|example)`

`eq|equals|all|{"field": "status", "op": "eq", "value": "active"}`

`neq|not equals|all|{"field": "status", "op": "neq", "value": "archived"}`

`gt|greater than|int, float, datetime, date|{"field": "priority", "op": "gt", "value": 1}`

`gte|greater than or equal|int, float, datetime, date|{"field": "due_date", "op": "gte", "value": "2021-01-01"}`

`lt|less than|int, float, datetime, date|{"field": "created_time", "op": "lt", "value": "2021-05-01T00:00:00Z"}`

`lte|less than or equal|int, float, datetime, date|{"field": "estimated_hours", "op": "lte", "value": 10}`

`in|set membership|all|{"field": "status", "op": "in", "value": ["open", "in_progress"]}`

`not_in|not in set|all|{"field": "status", "op": "not_in", "value": ["archived", "closed"]}`

`is_null|field is null|nullable fields|{"field": "assignee_user_id", "op": "is_null", "value": true}`

`is_not_null|field is not null|nullable fields|{"field": "due_date", "op": "is_not_null", "value": true}`

`starts_with|anchored prefix match|varchar|{"field": "name", "op": "starts_with", "value": "John"}`

contains_key|JSON object contains key|json|{"field": "monitor_data_json", "o
range|between two values|int, float, datetime, date|{"field": "priority", "op

Appendix U: Cross-Reference — Methods to Ops Book Concepts

method_to_book(method|book_concepts|book_rules|book_claims|book_distinction

M01 Entity Enumeration|C16 (Slicing the Pie), C17 (Comprehensive)|R5 (Slice t

M02 Field Enumeration|C8 (Data), C7 (Knowability)|R8 (Data Primacy)|K3 (Virtu

M04 Lifecycle Identification|C9 (Logic), C11 (System)|R8 (Data Primacy)|K7 (T
Logic outlives goal-Logic)|D2 (Data vs Logic)

M09 Architecture Position|C20 (Attribute Axis), C21 (Axiomatic Engineering)|R
9-0.9%)|K11 (Better not Best)|D5 (Best vs Better)

M10 Entity Design|C8 (Data), C17 (Comprehensive)|R8, R15 (Constraints On)|K4

M17 Draft Mode Config|C20 (Attribute Axis), C15 (Philosopher's Knife)|R1 (pri

M19 Approval Rules|C56 (AAA), C24 (Impersonal Decision)|R6 (AAA), R1 (priorit

M20 Access Control|C56 (AAA), C30 (Security Zone)|R6 (AAA), R9 (One Way)|K1 (C

M24 Runner Design|C36 (Idempotency), C31 (Operational Logic)|R3 (Idempotency)

M25 Puller Runner|C40 (Source of Truth), C41 (Knowing the Present)|R8 (Data Pr

M26 Reconciler Runner|C36 (Idempotency), C39 (Model for Control)|R3 (Idempote

M33 Server Runner|C35 (DOS), C34 (Distributed System)|R2 (0/1/Infinity)|K10 (

M39 Governed-State Pattern|C17 (Comprehensive), C2 (Control)|R5, R8, R9|K1, K

M44 AI Schema Generation|C8 (Data), C7 (Knowability)|R8 (Data Primacy)|K3, K4

M46 AI Change Proposer|C56 (AAA), C2 (Control)|R6 (AAA), R3 (Idempotency)|K1

[[HOWL-COMP-9-2026](#)] Building Applications with OpsDB Application Architecture.

Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-9-2026>

[[HOWL-COMP-1-2026](#)] Implementing a Tall-Infra Data-Only Execution System .
Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-1-2026>

[[HOWL-COMP-2-2026](#)] Tall-Infra, Data-Only Development. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP-2-2026>

[[HOWL-COMP-3-2026](#)] Silo OS. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-3-2026>

[[HOWL-COMP-4-2026](#)] Geometric Security. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-4-2026>

[[HOWL-COMP-5-2026](#)] Partial Geometric Security. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP-5-2026>

[[HOWL-COMP-6-2026](#)] SIML - Silo Markup Language (TOML Format). Github:
<https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-6-2026>

[[HOWL-COMP-7-2026](#)] SiQL. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-7-2026>

[[HOWL-COMP-8-2026](#)] SiloSec Protocol Specification. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP-8-2026>